

Sumário

1	Introdução aos Algoritmos e à Programação	7
1.1	Lógica de Programação	8
1.2	O que é um Algoritmo?	8
1.3	Algoritmos no Cotidiano	9
1.4	Características de um Algoritmo	9
1.4.1	Sequência organizada	9
1.4.2	Clareza	10
1.4.3	Finitude	10
1.4.4	Resultado	10
1.5	Algoritmos Computacionais	10
1.6	Algoritmo, Programa e Linguagem de Programação	11
1.7	Do Problema ao Programa	11
1.8	Formas de Representação de Algoritmos	12
1.8.1	Linguagem Natural	13
1.8.2	Fluxograma ou Diagrama de Blocos	13
1.8.3	Português Estruturado	14
1.9	Comparando as Formas de Representação	14
1.10	Primeiro Contato com o Portugol	15
1.10.1	Identificação do algoritmo	15
1.10.2	Declaração de dados	15
1.10.3	Seção de comandos	15
1.11	Nosso Primeiro Algoritmo	16
1.12	Comentários em um Algoritmo	16
1.13	Algoritmos e Resolução de Problemas	17
1.14	Resumo do Capítulo	17
1.15	Atividades	17
1.16	Desafio	19
2	Representação de Algoritmos	20
2.1	Formas de Representação de Algoritmos	21
2.2	Representação em Linguagem Natural	21
2.3	Fluxogramas ou Diagramas de Blocos	22
2.4	Principais Símbolos dos Fluxogramas	22
2.5	Representação Visual dos Símbolos	23
2.6	Fluxo de Execução	23
2.7	Exemplo de Fluxograma Sequencial	23
2.8	Símbolo de Decisão	24

2.9	Boas Práticas na Construção de Fluxogramas	25
2.10	Português Estruturado	26
2.11	Pseudocódigo e Linguagem de Programação	26
2.12	Portugol e VisuAlg	27
2.13	Do Fluxograma para o Portugol	27
2.14	Um Problema, Três Representações	28
2.14.1	Representação em linguagem natural	28
2.14.2	Representação por fluxograma	29
2.14.3	Representação em Portugol	29
2.15	Comparando as Representações	30
2.16	Diagrama de Nassi-Shneiderman	30
2.17	Estratégia para Construir um Algoritmo	30
2.18	Resumo do Capítulo	31
2.19	Atividades	31
2.20	Desafio	33
3	Elementos Fundamentais dos Algoritmos	34
3.1	Dados	35
3.2	Constantes	35
3.2.1	Constantes numéricas	35
3.2.2	Constantes de caracteres	36
3.2.3	Constantes lógicas	36
3.3	Variáveis	36
3.4	Variáveis e Memória	37
3.5	Tipos de Dados	37
3.5.1	Tipo inteiro	38
3.5.2	Tipo real	38
3.5.3	Tipo caractere	38
3.5.4	Tipo lógico	39
3.6	Identificadores (Nome)	39
3.7	Regras para Nomes de Variáveis	40
3.8	Palavras Reservadas	40
3.9	Declaração de Variáveis	41
3.10	Exemplo de Declaração	41
3.11	Atribuição	42
3.12	Atribuição Não é Igualdade Matemática	42
3.13	Atribuindo o Valor de Outra Variável	43
3.14	Atribuindo o Resultado de uma Expressão	44
3.15	Compatibilidade de Tipos	44
3.16	Erros Comuns	45
3.16.1	Utilizar uma variável sem declará-la	45
3.16.2	Utilizar um nome inválido	45
3.16.3	Utilizar espaço no identificador (nome variável)	45
3.16.4	Utilizar uma palavra reservada	45
3.16.5	Atribuir um valor incompatível	46
3.17	Exemplo Completo	46
3.18	Resumo do Capítulo	47

3.19	Atividades	47
3.20	Desafio	50
4	Operadores e Expressões	51
4.1	Operadores e Operandos	52
4.2	O que é uma Expressão?	53
4.3	Expressões Aritméticas	53
4.4	Operadores Aritméticos	54
4.5	Adição e Subtração	54
4.6	Multiplicação	54
4.7	Divisão	55
4.8	Divisão Inteira	55
4.9	Resto da Divisão	55
4.10	Potenciação	56
4.11	Linearização de Expressões	56
4.12	Precedência dos Operadores Aritméticos	57
4.13	Utilizando Parênteses	57
4.14	Exemplo Resolvido: Média	58
4.15	Operadores Relacionais	58
4.16	Principais Operadores Relacionais	59
4.17	Resultados das Expressões Relacionais	59
4.18	Atribuição e Comparação	60
4.19	Expressões Lógicas	60
4.20	Operadores Lógicos	60
4.21	Operador e	61
4.22	Tabela-Verdade do Operador e	61
4.23	Operador ou	61
4.24	Tabela-Verdade do Operador ou	62
4.25	Operador nao	62
4.26	Tabela-Verdade do Operador nao	62
4.27	Operador xou	62
4.28	Combinando Expressões Lógicas	63
4.29	Uso de Parênteses em Expressões Lógicas	64
4.30	Expressões Literais	64
4.31	Concatenação	64
4.32	Comparação entre Caracteres	65
4.33	Expressões Compostas	65
4.34	Avaliando uma Expressão Passo a Passo	66
4.35	Outro Exemplo Resolvido	66
4.36	Erros Comuns	67
4.36.1	Utilizar x para multiplicação	67
4.36.2	Confundir atribuição com comparação	67
4.36.3	Esquecer os parênteses	67
4.36.4	Esquecer aspas em textos	67
4.37	Resumo do Capítulo	68
4.38	Atividades	68
4.39	Desafio	71

5	Entrada, Processamento e Saída de Dados	72
5.1	O Modelo Entrada-Processamento-Saída	73
5.2	Representação Gráfica	73
5.3	Entrada de Dados	74
5.4	O Comando <code>leia</code>	74
5.5	Exemplo de Entrada	74
5.6	Orientando o Usuário	75
5.7	Lendo Vários Dados	75
5.8	Processamento	76
5.9	Exemplo de Processamento	76
5.10	Saída de Dados	77
5.11	O Comando <code>escreva</code>	77
5.12	O Comando <code>escreval</code>	77
5.13	<code>escreva</code> e <code>escreval</code>	78
5.14	Apresentando o Valor de uma Variável	78
5.15	Textos e Variáveis na Mesma Saída	79
5.16	Apresentando Expressões	79
5.17	Formatação Básica de Valores	79
5.18	Algoritmos Sequenciais	80
5.19	Exemplo 1 – Soma de Dois Números	80
5.20	Fluxograma do Exemplo	81
5.21	Exemplo 2 – Média de Duas Notas	81
5.22	Exemplo 3 – Área de um Retângulo	82
5.23	Exemplo 4 – Conversão de Temperatura	83
5.24	Exemplo 5 – Preço Final	83
5.25	Planejando Antes de Programar	84
5.26	Erros Comuns	84
5.26.1	Utilizar uma variável sem declaração	84
5.26.2	Esquecer a leitura	85
5.26.3	Confundir texto com variável	85
5.26.4	Não orientar a entrada	86
5.26.5	Utilizar uma fórmula incorreta	86
5.27	Estrutura Geral de um Algoritmo Sequencial	86
5.28	Resumo do Capítulo	87
5.29	Atividades	87
5.30	Desafio	89
6	Estruturas Condicionais	91
6.1	Tomada de Decisão	92
6.2	Condições	92
6.3	Estrutura Condicional Simples	93
6.4	Exemplo de Condicional Simples	93
6.5	Fluxograma da Condicional Simples	94
6.6	Exemplo 2 – Número Positivo	95
6.7	Estrutura Condicional Composta	95
6.8	Funcionamento da Condicional Composta	96
6.9	Exemplo 3 – Aprovação	96

6.10	Fluxograma da Condicional Composta	97
6.11	Exemplo 4 – Número Par ou Ímpar	97
6.12	Utilizando Operadores Lógicos nas Condições	98
6.13	Exemplo 5 – Valor Dentro de um Intervalo	98
6.14	Condicionais Aninhadas	99
6.15	Exemplo 6 – Situação do Estudante	99
6.16	Visualizando o Aninhamento	101
6.17	Indentação	101
6.18	Condicionais Independentes	102
6.19	Condicionais Encadeadas	102
6.20	Ordem das Condições	103
6.21	Seleção Múltipla	104
6.22	Exemplo 7 – Opções de um Menu	104
6.23	Vários Valores em um Mesmo Caso	105
6.24	se ou escolha?	105
6.25	Erros Comuns	106
6.25.1	Confundir atribuição com comparação	106
6.25.2	Esquecer o fimse	106
6.25.3	Escrever senao em uma condicional simples	107
6.25.4	Condições incompletas	107
6.25.5	Intervalos sobrepostos	107
6.26	Escolhendo a Estrutura Adequada	107
6.27	Resumo do Capítulo	108
6.28	Atividades	108
6.29	Desafio	111
7	Estruturas de Repetição	112
7.1	Laços de Repetição	113
7.2	Quando Utilizar uma Repetição?	113
7.3	Estruturas de Repetição no VisuAlg	114
7.4	Repetição com Teste no Início	114
7.5	Exemplo com enquanto	115
7.6	Entendendo o Funcionamento	115
7.7	Elementos de um Laço Controlado por Condição	116
7.8	Fluxograma do enquanto	117
7.9	O Bloco Pode Não Ser Executado	117
7.10	Exemplo – Contagem Regressiva	118
7.11	Laço Infinito	118
7.12	Repetição com Teste no Fim	119
7.13	A Condição do repita	119
7.14	Exemplo com repita	120
7.15	O Bloco é Executado pelo Menos uma Vez	120
7.16	Fluxograma do repita	121
7.17	Exemplo – Repetir até o Usuário Escolher Sair	121
7.18	Comparando enquanto e repita	122
7.19	Repetição com Variável de Controle	123
7.20	Exemplo com para	123

7.21	Variável de Controle	124
7.22	Utilizando passo	124
7.23	Contagem Decrescente com para	124
7.24	Exemplo – Tabuada	125
7.25	Quando Utilizar o para ?	125
7.26	Um Mesmo Problema com Diferentes Estruturas	126
7.27	Escolhendo a Estrutura de Repetição	127
7.28	Contadores	127
7.29	Exemplo – Quantidade de Aprovados	127
7.30	Por que Inicializar o Contador?	128
7.31	Acumuladores	129
7.32	Contador e Acumulador	129
7.33	Exemplo – Soma de Vários Valores	130
7.34	Calculando uma Média com Acumulador	130
7.35	Contador e Acumulador no Mesmo Algoritmo	131
7.36	Valor Sentinela	132
7.37	Leitura Antes e Dentro do Laço	132
7.38	Repetições Aninhadas	133
7.39	Entendendo a Repetição Aninhada	133
7.40	Exemplo – Tabuadas de 1 a 5	134
7.41	Erros Comuns	135
7.41.1	Esquecer de atualizar a variável de controle	135
7.41.2	Atualizar na direção errada	135
7.41.3	Inicializar o acumulador dentro do laço	136
7.41.4	Inicializar o contador dentro do laço	136
7.41.5	Confundir as condições de enquanto e repita	136
7.42	Comparação Geral das Estruturas	137
7.43	Resumo do Capítulo	137
7.44	Atividades	138
7.45	Desafio	141

Capítulo 1

Introdução aos Algoritmos e à Programação

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender o papel da lógica na resolução de problemas;
- definir o conceito de algoritmo;
- reconhecer algoritmos presentes em situações do cotidiano;
- diferenciar algoritmo, programa e linguagem de programação;
- identificar as principais formas de representação de algoritmos;
- compreender o papel dos fluxogramas e do Português Estruturado;
- reconhecer a estrutura básica de um algoritmo escrito em Portugal;
- compreender a relação entre problema, algoritmo e programa.

Os computadores estão presentes em muitas atividades do cotidiano. Eles são utilizados para realizar cálculos, organizar informações, controlar equipamentos, acessar serviços, processar imagens, realizar pagamentos, permitir a comunicação entre pessoas e executar diversas outras tarefas.

Entretanto, um computador não resolve um problema simplesmente porque ele foi apresentado à máquina. Para que uma determinada tarefa seja realizada, é necessário descrever de maneira organizada quais ações deverão ser executadas. Nesse contexto, surge a importância dos **algoritmos**, processo diretamente relacionado ao estudo da **lógica de programação**.

Ideia central

Programar não consiste apenas em memorizar comandos de uma linguagem. Antes de escrever um programa, é necessário compreender o problema e construir uma estratégia para resolvê-lo.

1.1 Lógica de Programação

A palavra *lógica* está relacionada à organização do raciocínio. No contexto da programação, a lógica permite estruturar pensamentos e ações de maneira que uma determinada tarefa possa ser realizada corretamente.

Podemos entender a **lógica de programação** como a utilização organizada do raciocínio para construir uma sequência de instruções destinada à resolução de um problema.

Essa sequência deve indicar claramente o que precisa ser feito e em que ordem cada ação deverá ocorrer. Considere, por exemplo, a tarefa de acessar um computador disponível em um laboratório escolar. Uma possível sequência de ações seria:

1. ligar o computador;
2. aguardar a inicialização do sistema;
3. informar o usuário;
4. informar a senha;
5. confirmar o acesso;
6. aguardar a abertura do ambiente de trabalho.

Observe que a **ordem das ações é importante**. Não seria possível informar a senha antes que o sistema estivesse preparado para recebê-la.

Assim, uma solução computacional normalmente envolve não apenas a identificação das ações necessárias, mas também a definição da ordem em que essas ações devem ser executadas.

1.2 O que é um Algoritmo?

Um **algoritmo** pode ser compreendido como uma sequência finita e organizada de instruções utilizada para realizar uma tarefa ou resolver um problema (**pereira2011visualg**).

A ideia de algoritmo não pertence exclusivamente à Computação. Diversas atividades realizadas no cotidiano podem ser descritas por meio de uma sequência ordenada de passos. Receitas culinárias, instruções para montagem de equipamentos, procedimentos para utilização de aparelhos e orientações para chegar a determinado local são exemplos de situações que apresentam características semelhantes às de um algoritmo (**carvalho2007algoritmos**).

Definição

Um **algoritmo** é uma sequência finita e organizada de instruções que descreve como realizar uma tarefa ou resolver determinado problema.

É importante observar que um algoritmo representa **um caminho para a solução**. Um mesmo problema pode possuir diferentes algoritmos capazes de produzir o mesmo resultado. Por exemplo, diferentes pessoas podem utilizar caminhos distintos para chegar à escola. Os caminhos podem ser diferentes, mas todos podem alcançar o mesmo objetivo. Em programação ocorre algo semelhante: um mesmo problema pode ser resolvido por algoritmos diferentes.

1.3 Algoritmos no Cotidiano

Os algoritmos fazem parte de muitas atividades diárias, mesmo quando não utilizamos essa palavra para descrevê-las. Considere a tarefa de preparar um copo de suco utilizando um liquidificador.

Exemplo 1 – Preparar um suco

1. separar a fruta;
2. colocar a fruta no liquidificador;
3. adicionar água;
4. adicionar açúcar, se necessário;
5. fechar o liquidificador;
6. ligar o liquidificador;
7. aguardar a mistura dos ingredientes;
8. desligar o liquidificador;
9. servir o suco.

Essa descrição apresenta uma sequência de passos destinada a alcançar um objetivo. Entretanto, algumas instruções ainda podem apresentar ambiguidades. Por exemplo, “**adicionar água**” não especifica a quantidade de água utilizada. Para uma pessoa, essa informação pode ser facilmente interpretada utilizando experiência e conhecimento prévio. Um computador, entretanto, exige instruções muito mais precisas.

Atenção

Computadores executam as instruções que recebem. Portanto, quanto mais precisa for a descrição de uma solução, menor será a possibilidade de interpretações incorretas.

1.4 Características de um Algoritmo

Para fins didáticos, podemos considerar algumas características importantes na elaboração de algoritmos.

1.4.1 Sequência organizada

As instruções devem estar organizadas em uma ordem coerente. Considere:

ligar o computador → iniciar o sistema → executar o programa

A alteração da ordem dessas ações pode impedir a realização da tarefa.

1.4.2 Clareza

Cada instrução deve indicar claramente a ação que deverá ser realizada. Uma instrução como:

“faça alguma coisa com os números”

não apresenta precisão suficiente. Seria mais adequado escrever:

“some os dois números informados”.

1.4.3 Finitude

Um algoritmo deve possuir um número finito de passos e, quando executado corretamente, deve chegar ao seu término. Posteriormente estudaremos estruturas que permitem repetir instruções. Mesmo nessas situações, deverá existir alguma condição que determine quando a repetição será encerrada.

1.4.4 Resultado

A execução de um algoritmo tem como objetivo produzir algum resultado relacionado ao problema que está sendo resolvido. Esse resultado pode ser um número, uma mensagem, uma classificação, uma alteração em um conjunto de dados ou qualquer outra informação necessária à solução.

1.5 Algoritmos Computacionais

Quando um algoritmo é desenvolvido com o objetivo de ser executado por um computador, falamos em um **algoritmo computacional**. Em um algoritmo computacional, as instruções devem ser descritas de modo que possam ser posteriormente representadas em uma linguagem de programação. Considere o seguinte problema:

Calcular a média de duas notas de um estudante.

Antes de escrever qualquer código, podemos organizar a solução da seguinte maneira:

1. obter a primeira nota;
2. obter a segunda nota;
3. somar as duas notas;
4. dividir o resultado por dois;
5. apresentar a média calculada.

Essa sequência constitui uma solução algorítmica para o problema. Observe que ainda não estamos preocupados com comandos específicos de Java, Python, C ou qualquer outra linguagem de programação. Estamos inicialmente interessados na **lógica da solução**.

1.6 Algoritmo, Programa e Linguagem de Programação

Algoritmo e programa são conceitos relacionados, mas não representam exatamente a mesma coisa. Um algoritmo descreve uma estratégia para resolver um problema. Um programa, por sua vez, representa a implementação dessa estratégia por meio de uma linguagem que possa ser processada pelo computador.

Tabela 1.1: Diferença entre algoritmo, programa e linguagem de programação.

Conceito	Descrição
Algoritmo	Sequência organizada de instruções utilizada para resolver um problema ou realizar uma tarefa.
Programa	Implementação de um algoritmo utilizando uma linguagem de programação.
Linguagem de programação	Conjunto de símbolos, palavras, estruturas e regras utilizado para escrever programas de computador.

Linguagens como C, Java, Python e outras fornecem formas específicas para representar as instruções que serão executadas pelo computador. Cada linguagem possui sua própria **sintaxe**, isto é, um conjunto de regras que determina como os comandos devem ser escritos. Nesse sentido, aprender programação envolve pelo menos dois conhecimentos diferentes:

- aprender a construir uma solução lógica para o problema;
- aprender a representar essa solução utilizando uma linguagem.

O estudo inicial de algoritmos concentra-se principalmente no primeiro desses aspectos.

1.7 Do Problema ao Programa

O desenvolvimento de um programa pode ser compreendido como um processo composto por diferentes etapas. Primeiramente, é necessário compreender o problema. Em seguida, deve-se elaborar uma estratégia de solução. Essa estratégia pode ser representada por um algoritmo e, depois, transformada em um programa. A Figura 1.1 apresenta uma representação simplificada desse processo.

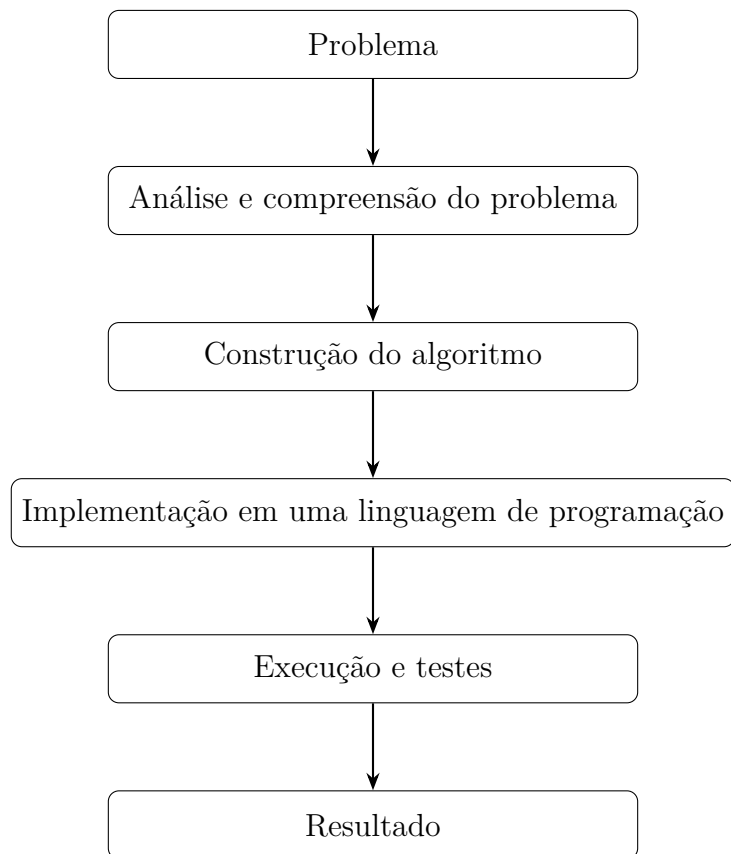


Figura 1.1: Representação simplificada do processo entre problema e programa.

Na prática, esse processo nem sempre ocorre de maneira completamente linear. Durante os testes, por exemplo, pode-se descobrir que o algoritmo apresenta algum problema. Nesse caso, será necessário retornar às etapas anteriores, corrigir a solução e realizar novos testes.

Importante

Um erro encontrado durante a execução de um programa pode ter sido causado pela implementação, mas também pode ter origem na própria estratégia utilizada para resolver o problema.

1.8 Formas de Representação de Algoritmos

Um algoritmo pode ser representado de diferentes maneiras. Entre as formas utilizadas no ensino de programação encontram-se a descrição em linguagem natural, as representações gráficas e o Português Estruturado ([carvalho2007algoritmos](#)). Nesta apostila serão utilizadas principalmente três formas:

- linguagem natural;
- fluxograma ou diagrama de blocos;
- Português Estruturado, também denominado pseudocódigo ou Portugol.

1.8.1 Linguagem Natural

A linguagem natural utiliza palavras e frases semelhantes às utilizadas normalmente pelas pessoas. Por exemplo, considere novamente o problema de calcular a média de duas notas:

1. solicitar a primeira nota;
2. solicitar a segunda nota;
3. calcular a média;
4. apresentar o resultado.

Essa forma possui a vantagem de ser simples e de fácil compreensão. Entretanto, a linguagem natural também pode apresentar ambiguidades. Por esse motivo, à medida que os problemas se tornam mais complexos, outras formas de representação podem facilitar a organização da solução.

1.8.2 Fluxograma ou Diagrama de Blocos

O **fluxograma**, também chamado de **diagrama de fluxo** ou **diagrama de blocos**, é uma representação gráfica de um algoritmo.

Nele, formas geométricas são utilizadas para representar diferentes tipos de ações, e setas indicam o fluxo de execução **carvalho2007algoritmos**. Considere o exemplo da média de duas notas.

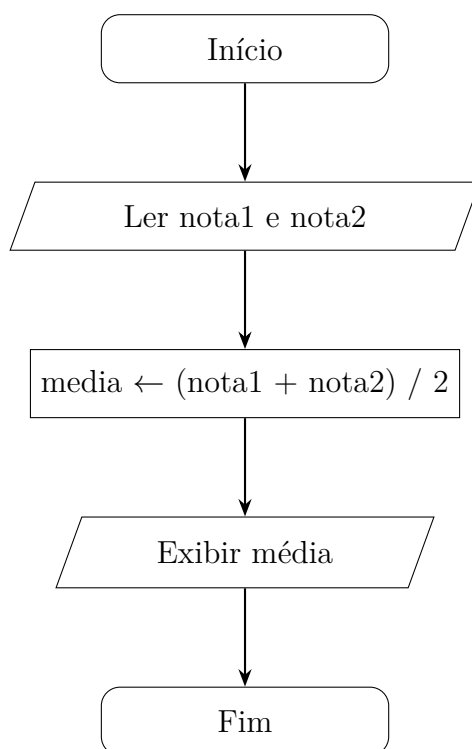


Figura 1.2: Fluxograma para o cálculo da média de duas notas.

O fluxograma permite visualizar de forma gráfica a sequência das operações realizadas pelo algoritmo. No próximo capítulo, os principais símbolos utilizados nos diagramas de blocos serão estudados com mais detalhes.

1.8.3 Português Estruturado

O **Português Estruturado** utiliza um conjunto limitado de palavras e estruturas para representar algoritmos. Essa forma de representação também é conhecida como **pseudocódigo**, **pseudolinguagem** ou **Portugol**.

O pseudocódigo funciona como uma representação intermediária entre a linguagem natural e uma linguagem de programação, permitindo que o estudante concentre inicialmente sua atenção na lógica utilizada para resolver o problema ([carvalho2007algoritmos](#)). O mesmo algoritmo da média poderia ser representado da seguinte maneira:

```
algoritmo "calcular_media"

var
    nota1, nota2, media: real

inicio

    leia(nota1)
    leia(nota2)

    media <- (nota1 + nota2) / 2

    escreval("Media = ", media)

fimalgoritmo
```

Não é necessário compreender todos os comandos desse exemplo neste momento. Cada elemento será estudado progressivamente nos próximos capítulos.

1.9 Comparando as Formas de Representação

O mesmo problema pode ser representado de maneiras diferentes, sem que sua lógica seja alterada. A Tabela 1.2 resume as representações estudadas.

Tabela 1.2: Principais formas de representação utilizadas nesta apostila.

Representação	Característica	Principal vantagem
Linguagem natural	Utiliza frases e palavras do cotidiano.	Facilidade inicial de compreensão.
Fluxograma	Representa graficamente o fluxo do algoritmo.	Facilita a visualização da sequência de ações.
Português Estruturado	Utiliza comandos organizados de forma semelhante às linguagens de programação.	Facilita a transição entre a lógica e a programação.

1.10 Primeiro Contato com o Portugol

Nesta apostila utilizaremos o **Portugol** como principal forma textual de representação dos algoritmos. Para permitir que os algoritmos sejam executados e testados, adotaremos como referência a sintaxe utilizada pelo **VisuAlg**. A estrutura básica de um algoritmo no VisuAlg pode ser representada da seguinte forma (pereira2011visualg):

```
algoritmo "nome_do_algoritmo"

var
    // declaracao das variaveis

inicio

    // comandos do algoritmo

finalgoritmo
```

Podemos dividir essa estrutura em três partes principais.

1.10.1 Identificação do algoritmo

A instrução

```
algoritmo "nome_do_algoritmo"
```

identifica o início do algoritmo e atribui um nome a ele. Por exemplo:

```
algoritmo "calcula_da_media"
```

1.10.2 Declaração de dados

A palavra

```
var
```

indica a região destinada à declaração das variáveis utilizadas pelo algoritmo. O conceito de variável será estudado detalhadamente em um capítulo posterior.

1.10.3 Seção de comandos

A palavra

```
inicio
```

marca o início das instruções que serão executadas. O término do algoritmo é indicado por:

```
finalgoritmo
```

1.11 Nosso Primeiro Algoritmo

Vamos construir um algoritmo muito simples cuja única função é apresentar uma mensagem na tela.

```
algoritmo "primeiro_algoritmo"
```

```
inicio
```

```
    escreval("Ola, mundo!")
```

```
fimalgoritmo
```

Quando esse algoritmo for executado, será apresentada a mensagem:

Ola, mundo!

O comando

```
escreval()
```

é utilizado no VisuAlg para apresentar informações na tela. Neste momento, nosso objetivo não é estudar detalhadamente esse comando, mas compreender a estrutura geral do algoritmo.

Observe

As palavras utilizadas pelo VisuAlg, como `algoritmo`, `inicio` e `fimalgoritmo`, possuem significado especial e fazem parte da sintaxe da linguagem.

1.12 Comentários em um Algoritmo

Comentários são textos utilizados para explicar partes do algoritmo. No VisuAlg, um comentário de linha é iniciado pelos caracteres:

```
//
```

Por exemplo:

```
algoritmo "exemplo"
```

```
// Este algoritmo apresenta uma mensagem na tela
```

```
inicio
```

```
    escreval("Bem-vindo ao estudo de algoritmos!")
```

```
fimalgoritmo
```

O comentário serve para auxiliar a leitura e a documentação do algoritmo e não representa uma instrução a ser executada pelo computador. O uso adequado de comentários pode tornar algoritmos maiores mais fáceis de compreender.

1.13 Algoritmos e Resolução de Problemas

Aprender algoritmos exige prática. Ler exemplos é importante para compreender conceitos, mas o desenvolvimento do raciocínio algorítmico ocorre principalmente quando o estudante tenta construir suas próprias soluções.

Ao receber um problema, algumas perguntas podem ajudar no desenvolvimento da solução:

Perguntas para orientar a resolução de problemas

1. Qual problema precisa ser resolvido?
2. Quais informações são necessárias?
3. Qual resultado deve ser produzido?
4. Quais operações precisam ser realizadas?
5. Em qual ordem essas operações devem ocorrer?
6. Como verificar se o resultado está correto?

Essas perguntas serão retomadas diversas vezes ao longo desta apostila.

1.14 Resumo do Capítulo

Neste capítulo vimos que a programação de computadores envolve muito mais do que escrever comandos. Antes de implementar um programa, é necessário compreender o problema e organizar uma estratégia para resolvê-lo.

Um algoritmo representa uma sequência organizada e finita de instruções destinada à realização de uma tarefa ou à solução de um problema. Os algoritmos podem ser representados por meio de diferentes formas. Nesta apostila serão utilizadas principalmente a linguagem natural, os fluxogramas e o Português Estruturado.

Também tivemos um primeiro contato com o Portugol e com a estrutura utilizada pelo VisuAlg, composta principalmente pelas palavras `algoritmo`, `var`, `inicio` e `fimalgoritmo`.

Nos próximos capítulos esses elementos serão aprofundados progressivamente.

1.15 Atividades

Questões conceituais

1. Explique, com suas próprias palavras, o que é lógica de programação.
2. O que é um algoritmo?
3. Por que um algoritmo deve apresentar suas instruções em uma ordem organizada?
4. Um mesmo problema pode possuir mais de um algoritmo capaz de resolvê-lo? Explique.

5. Qual é a diferença entre algoritmo e programa?
6. O que é uma linguagem de programação?
7. Cite três exemplos de algoritmos presentes no cotidiano.
8. O que significa dizer que um algoritmo deve ser finito?
9. Qual é a função de um fluxograma?
10. O que é Português Estruturado?
11. Qual é a relação entre Português Estruturado, pseudocódigo e Portugol?
12. Qual é a finalidade dos comentários em um algoritmo?

Atividades práticas

1. Escreva, utilizando linguagem natural, um algoritmo para escovar os dentes.
2. Escreva um algoritmo para ligar um computador e acessar o sistema.
3. Escreva um algoritmo para preparar um sanduíche.
4. Analise o algoritmo produzido na questão anterior e identifique instruções que poderiam apresentar ambiguidades.
5. Organize as ações seguintes na ordem correta:

executar o programa; compreender o problema; testar a solução; escrever o algoritmo; implementar o programa.

6. Represente por meio de um fluxograma o procedimento:

início → ler um número → mostrar o número → fim.

7. Digite e execute no VisuAlg o seguinte algoritmo:

```
algoritmo "mensagem"
```

```
inicio
```

```
    escreval("Estou aprendendo algoritmos!")
```

```
fimalgoritmo
```

Explique o que ocorre durante sua execução.

8. Modifique o algoritmo anterior para apresentar seu nome, curso e turma.

1.16 Desafio

Desafio – Pense como um programador

Uma pessoa precisa utilizar um caixa eletrônico para consultar o saldo de sua conta. Descreva um algoritmo em linguagem natural contendo todas as ações necessárias desde o momento em que a pessoa se aproxima do equipamento até o encerramento da operação. Depois de escrever o algoritmo, analise cada instrução e responda:

1. As instruções estão organizadas na ordem correta?
2. Existe alguma instrução ambígua?
3. Alguma etapa importante foi esquecida?
4. O algoritmo possui início e término claramente definidos?

Capítulo 2

Representação de Algoritmos

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender por que um algoritmo pode ser representado de diferentes formas;
- descrever soluções utilizando linguagem natural;
- reconhecer os principais símbolos utilizados em fluxogramas;
- interpretar o fluxo de execução de um diagrama de blocos;
- construir fluxogramas simples;
- compreender o conceito de Português Estruturado;
- reconhecer os termos pseudocódigo, pseudolinguagem e Portugol;
- representar uma mesma solução por meio de linguagem natural, fluxograma e Portugol;
- relacionar uma representação gráfica com sua correspondente representação textual.

No capítulo anterior, vimos que um algoritmo descreve uma sequência organizada de instruções para realizar uma tarefa ou resolver um problema.

Entretanto, a solução de um problema pode ser registrada de diferentes formas. Podemos, por exemplo, utilizar frases em língua portuguesa, desenhos formados por símbolos e setas ou uma linguagem textual mais próxima das linguagens de programação. Essas diferentes maneiras são chamadas de **formas de representação de algoritmos**.

A utilização de diferentes representações é importante porque permite observar uma mesma solução sob perspectivas distintas. Uma descrição textual pode facilitar o primeiro entendimento do problema, enquanto uma representação gráfica pode evidenciar melhor o fluxo das ações. O Português Estruturado, por sua vez, aproxima a solução lógica da forma utilizada em uma linguagem de programação.

Ideia central

O algoritmo representa a **solução**. A linguagem natural, o fluxograma e o Portugol são diferentes maneiras de **representar essa solução**.

2.1 Formas de Representação de Algoritmos

Existem diferentes maneiras de representar um algoritmo. Entre elas estão descrições utilizando uma língua natural, representações gráficas e linguagens textuais estruturadas ([carvalho2007algoritmos](#)). Nesta apostila utilizaremos principalmente:

1. **Linguagem Natural;**
2. **Fluxograma ou Diagrama de Blocos;**
3. **Português Estruturado ou Portugol.**

Cada representação possui características próprias e pode ser mais adequada em determinadas situações. Ao longo da disciplina, frequentemente apresentaremos uma solução primeiro em forma gráfica e, posteriormente, em Portugol.

2.2 Representação em Linguagem Natural

A forma mais simples de representar inicialmente um algoritmo consiste em utilizar a própria linguagem utilizada pelas pessoas no cotidiano. Considere o seguinte problema:

Elaborar um algoritmo para calcular a soma de dois números.

Uma possível solução em linguagem natural seria:

1. solicitar o primeiro número;
2. solicitar o segundo número;
3. somar os dois números;
4. apresentar o resultado da soma.

A vantagem dessa representação é sua simplicidade. Entretanto, a linguagem natural pode apresentar **ambiguidades**. Uma mesma frase pode ser interpretada de maneiras diferentes por pessoas distintas. Considere a instrução:

“Calcule o resultado utilizando os valores informados.”

Não está claro qual operação deverá ser realizada. Uma descrição melhor seria:

“Some os dois valores informados e apresente o resultado.”

Atenção

Uma boa descrição algorítmica deve evitar instruções vagas como “faça o cálculo”, “processe os dados” ou “verifique o resultado” quando não estiver claro qual operação deverá ser realizada.

2.3 Fluxogramas ou Diagramas de Blocos

O **fluxograma**, também conhecido como **diagrama de fluxo** ou **diagrama de blocos**, é uma representação gráfica de um algoritmo.

Nessa representação, diferentes formas geométricas são utilizadas para indicar ações, entrada e saída de dados, decisões e outros elementos do algoritmo. Esses símbolos são conectados por setas que indicam a direção do fluxo de execução (**carvalho2007algoritmos**). O fluxograma é particularmente útil para compreender visualmente:

- onde o algoritmo começa;
- qual ação deve ser executada;
- em qual ordem as ações ocorrem;
- onde dados são recebidos ou apresentados;
- onde ocorre uma decisão;
- quais caminhos podem ser seguidos;
- onde o algoritmo termina.

2.4 Principais Símbolos dos Fluxogramas

Nesta apostila utilizaremos um conjunto básico de símbolos suficiente para representar os algoritmos estudados inicialmente.

Tabela 2.1: Principais símbolos utilizados nos fluxogramas.

Símbolo	Nome	Finalidade
Início / Fim	Terminal	Indica o início ou o encerramento do algoritmo.
Retângulo	Processamento	Representa uma operação ou processamento, como um cálculo ou uma atribuição.
Paralelogramo	Entrada / Saída	Representa a entrada de dados ou a apresentação de informações.
Losango	Decisão	Representa uma condição que pode produzir caminhos diferentes no algoritmo.
Seta	Fluxo	Indica a sequência e a direção de execução das operações.
Círculo	Conector	Pode ser utilizado para conectar partes de um fluxograma, especialmente em diagramas maiores.

Os símbolos serão apresentados gradualmente. Nos primeiros algoritmos, utilizaremos principalmente **início/fim**, **entrada/saída** e **processamento**.

2.5 Representação Visual dos Símbolos

A Figura 2.1 apresenta graficamente os principais símbolos que serão utilizados.

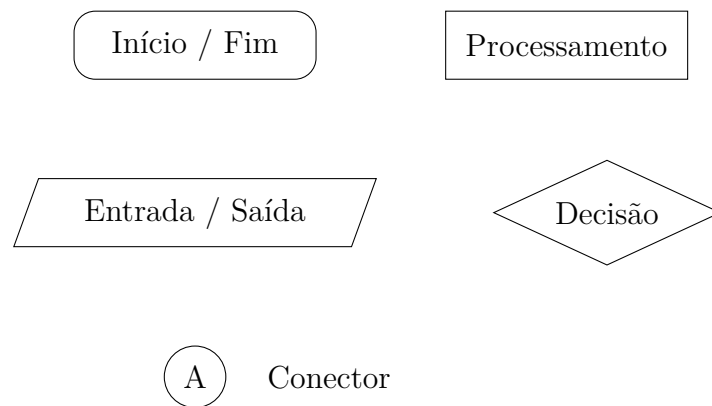


Figura 2.1: Principais símbolos utilizados em diagramas de blocos.

2.6 Fluxo de Execução

As setas utilizadas em um fluxograma indicam a ordem em que as operações são executadas. Considere a sequência:

Início → **Entrada** → **Processamento** → **Saída** → **Fim**

Isso significa que cada etapa é executada após a conclusão da etapa anterior. Em algoritmos simples, o fluxo normalmente ocorre de cima para baixo.

Entretanto, estruturas de decisão e repetição poderão alterar esse fluxo. Esses casos serão estudados detalhadamente nos capítulos seguintes.

2.7 Exemplo de Fluxograma Sequencial

Considere novamente o problema:

Ler dois números, calcular sua soma e apresentar o resultado.

Primeiramente podemos identificar três etapas fundamentais:

Entrada: dois números;

Processamento: soma dos números;

Saída: resultado da soma.

O fluxograma correspondente é apresentado na Figura 2.2.

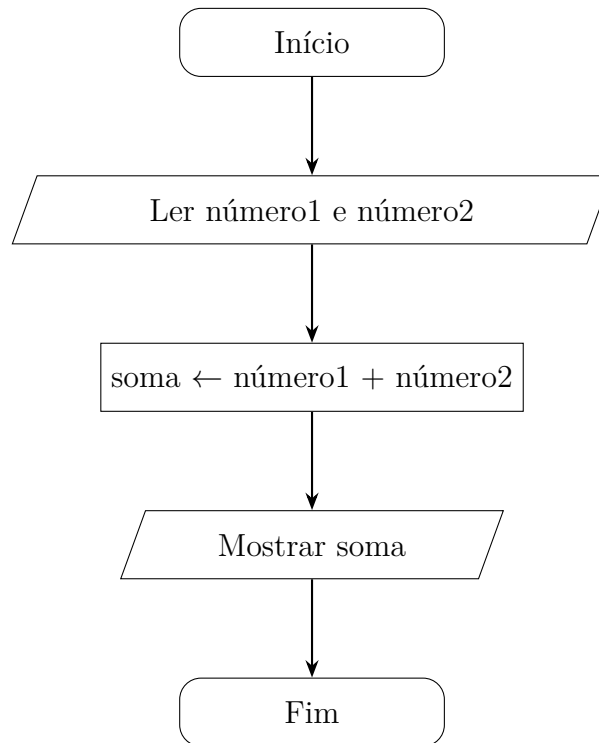


Figura 2.2: Fluxograma para calcular a soma de dois números.

Observe que cada símbolo representa uma função específica. O algoritmo começa no símbolo de início, recebe os dois valores, realiza o processamento, apresenta o resultado e termina.

2.8 Símbolo de Decisão

Nem todos os algoritmos seguem sempre um único caminho. Em determinadas situações, o próximo passo depende do resultado de uma condição. Considere o seguinte problema:

Ler a idade de uma pessoa e informar se ela possui 18 anos ou mais.

Nesse caso, precisamos realizar uma pergunta lógica:

$$\text{idade} \geq 18?$$

Existem duas possibilidades:

- **Sim:** apresentar “18 anos ou mais”;
- **Não:** seguir diretamente para o final.

O losango representa esse tipo de decisão.

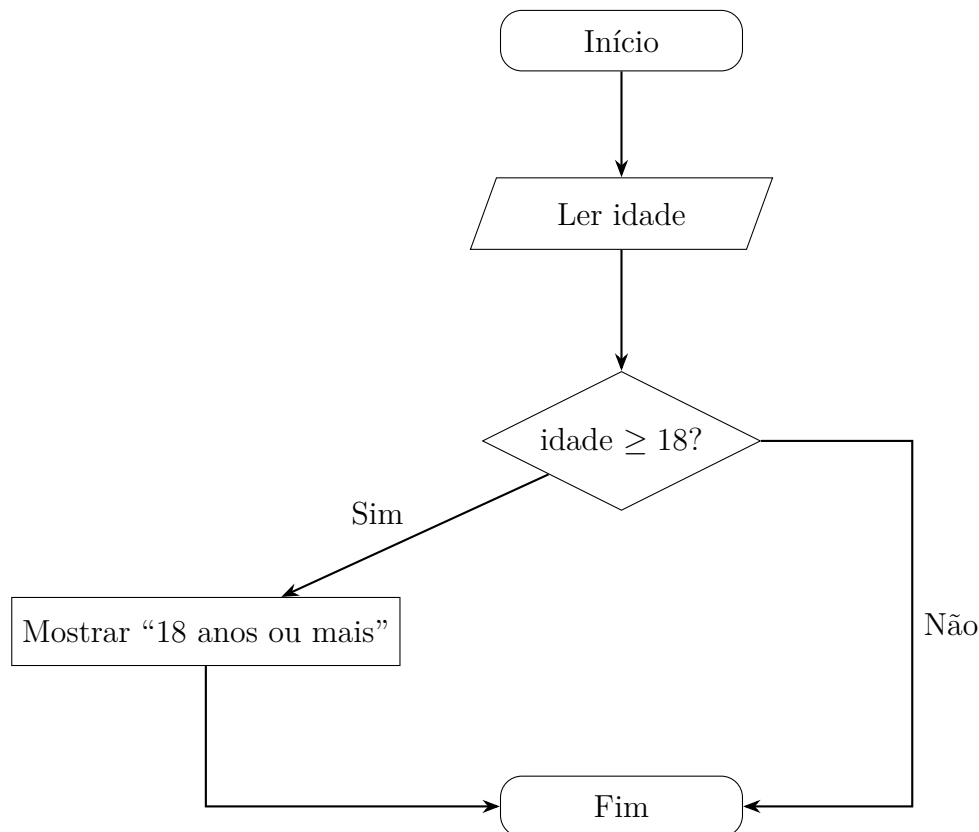


Figura 2.3: Exemplo de decisão em um fluxograma.

As estruturas de decisão serão estudadas detalhadamente no capítulo sobre estruturas condicionais.

2.9 Boas Práticas na Construção de Fluxogramas

Algumas práticas tornam um fluxograma mais fácil de compreender.

1. Utilize um único ponto claramente identificado como início.
2. Indique claramente o término do algoritmo.
3. Mantenha o fluxo principal preferencialmente de cima para baixo.
4. Utilize setas para indicar a direção da execução.
5. Evite cruzamentos desnecessários entre linhas.
6. Escreva instruções curtas dentro dos símbolos.
7. Utilize o símbolo adequado para cada tipo de operação.
8. Identifique os caminhos de uma decisão, normalmente utilizando indicações como **Sim** e **Não**.
9. Em fluxogramas maiores, utilize conectores quando isso melhorar a organização visual.

Importante

O objetivo do fluxograma não é tornar o algoritmo mais complicado, mas tornar seu fluxo mais fácil de visualizar.

2.10 Português Estruturado

Outra forma importante de representar algoritmos é o **Português Estruturado**. O Português Estruturado utiliza palavras da língua portuguesa organizadas segundo uma estrutura definida para representar as instruções de um algoritmo (carvalho2007algoritmos). Também podem ser encontrados os termos:

- pseudocódigo;
- pseudolinguagem;
- Portugol.

Embora existam algumas diferenças entre materiais didáticos e ferramentas, esses termos são frequentemente utilizados no ensino introdutório de algoritmos.

O principal objetivo do pseudocódigo é permitir que o estudante represente uma solução utilizando uma estrutura semelhante à de uma linguagem de programação, sem concentrar inicialmente sua atenção em todos os detalhes sintáticos de uma linguagem profissional.

2.11 Pseudocódigo e Linguagem de Programação

Uma linguagem de programação possui regras sintáticas que precisam ser obedecidas com precisão para que o programa possa ser processado. No pseudocódigo, o interesse principal está na representação da **lógica da solução**.

Ferrari e Cechinel destacam que o uso de pseudocódigo permite trabalhar estruturas algorítmicas sem que a preocupação principal esteja em detalhes de sintaxe específicos de determinada linguagem (ferrariCechinelAlgoritmos). Considere a ideia:

Ler dois números, somá-los e apresentar o resultado.

Em pseudocódigo, poderíamos escrever:

inicio

leia(numero1)

leia(numero2)

soma <- numero1 + numero2

escreva(soma)

fim

Essa escrita está mais próxima de um programa do que a descrição em linguagem natural, mas ainda possui forte relação com a forma como descrevemos o problema.

2.12 Portugol e VisuAlg

Nesta apostila utilizaremos principalmente a sintaxe do **VisuAlg** para representar e executar os algoritmos. Um algoritmo básico possui a seguinte estrutura:

```
algoritmo "nome_do_algoritmo"

var
    // declaracao de variaveis

inicio

    // comandos

finalgoritmo
```

Neste momento, ainda não precisamos compreender todos esses elementos. Nos capítulos seguintes serão estudados:

- constantes;
- variáveis;
- tipos de dados;
- atribuição;
- entrada e saída;
- expressões;
- estruturas condicionais;
- estruturas de repetição.

2.13 Do Fluxograma para o Portugol

Uma das habilidades que desenvolveremos ao longo desta apostila é transformar uma representação gráfica em uma representação textual. Considere novamente o fluxograma da soma apresentado anteriormente. Sua representação em Portugol pode ser escrita da seguinte maneira:

```
algoritmo "soma"

var
    numero1, numero2, soma: real

inicio
```

```
leia(numero1)
leia(numero2)

soma <- numero1 + numero2

escreval("Resultado = ", soma)

finalgoritmo
```

Podemos estabelecer a seguinte correspondência:

Tabela 2.2: Correspondência entre fluxograma e Portugol.

Fluxograma	Portugol
Início	inicio
Entrada de dados	leia(...)
Processamento	expressão ou atribuição
Saída de dados	escreva(...) ou escreval(...)
Decisão	estrutura se
Fim	finalgoritmo

Essa correspondência será retomada diversas vezes nos próximos capítulos.

2.14 Um Problema, Três Representações

Vamos observar o mesmo problema utilizando três representações diferentes. Considere:

Calcular o dobro de um número informado pelo usuário.

2.14.1 Representação em linguagem natural

1. solicitar um número;
2. multiplicar o número por dois;
3. apresentar o resultado.

2.14.2 Representação por fluxograma

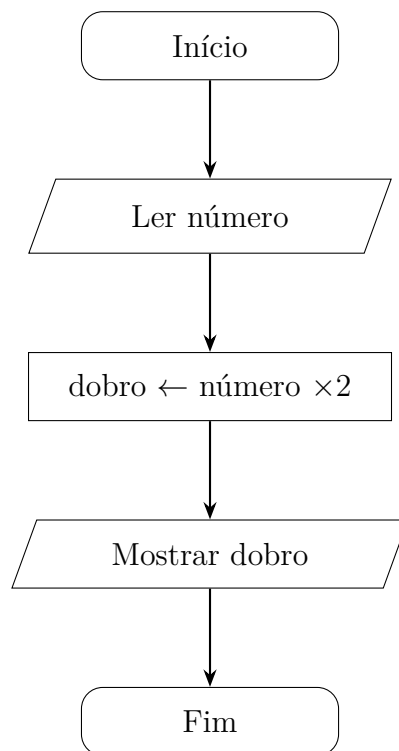


Figura 2.4: Fluxograma para calcular o dobro de um número.

2.14.3 Representação em Portugol

```
algoritmo "dobro"

var
    numero, dobro: real

inicio

    leia(numero)

    dobro <- numero * 2

    escreval("Dobro = ", dobro)

finalgoritmo
```

Apesar de visualmente diferentes, as três representações descrevem exatamente a mesma sequência lógica.

2.15 Comparando as Representações

Cada forma de representação apresenta vantagens e limitações.

Tabela 2.3: Comparação entre formas de representação de algoritmos.

Representação	Vantagens	Limitações
Linguagem natural	Simples e familiar para iniciantes.	Pode apresentar ambiguidades e tornar-se extensa.
Fluxograma	Facilita a visualização do fluxo e das decisões.	Pode tornar-se grande em problemas complexos.
Português Estruturado	Aproxima a solução de uma linguagem de programação e facilita a implementação.	Exige o aprendizado de uma organização e de comandos próprios.

Não existe uma representação que seja obrigatoriamente melhor em todas as situações. Para fins didáticos, utilizaremos o fluxograma principalmente para visualizar a lógica e o Portugol para descrever e executar os algoritmos.

2.16 Diagrama de Nassi–Shneiderman

Outra forma gráfica encontrada na literatura é o **Diagrama de Nassi–Shneiderman**, também conhecido como **Diagrama de Chapin**.

Nessa representação, as instruções são organizadas dentro de blocos retangulares subdivididos, evidenciando a estrutura hierárquica do algoritmo (carvalho2007algoritmos).

Embora seja uma técnica importante historicamente e ainda utilizada em alguns materiais de ensino, nesta apostila adotaremos prioritariamente o **fluxograma** e o **Portugol**. O objetivo é manter uma relação direta entre:

Problema → Fluxograma → Portugol

2.17 Estratégia para Construir um Algoritmo

Antes de desenhar um fluxograma ou escrever um algoritmo em Portugol, é recomendável identificar os elementos básicos do problema. Uma estratégia inicial consiste em responder três perguntas:

1. **Quais dados serão fornecidos?**
2. **O que precisa ser feito com esses dados?**
3. **Qual resultado deverá ser apresentado?**

Essas perguntas correspondem ao modelo:

Entrada → Processamento → Saída

Considere o problema:

Ler o preço de um produto e calcular o preço final acrescentando R\$ 10,00.

Podemos identificar:

Entrada: preço do produto;

Processamento: adicionar 10 ao preço;

Saída: preço final.

Essa decomposição facilita tanto a construção do fluxograma quanto a escrita posterior do algoritmo.

2.18 Resumo do Capítulo

Neste capítulo estudamos diferentes maneiras de representar algoritmos. A linguagem natural utiliza frases próximas da comunicação cotidiana, sendo útil principalmente durante a compreensão inicial do problema.

O fluxograma representa graficamente o algoritmo utilizando símbolos e setas. Entre os principais elementos utilizados estão os símbolos de início e fim, processamento, entrada e saída, decisão e fluxo.

O Português Estruturado é uma representação textual intermediária entre a linguagem natural e uma linguagem de programação. Também pode ser denominado pseudocódigo, pseudolinguagem ou Portugol.

Nesta apostila utilizaremos principalmente fluxogramas e Portugol. Dessa forma, uma mesma solução poderá ser inicialmente visualizada por meio de um diagrama e, posteriormente, representada de maneira textual utilizando a sintaxe do VisuAlg.

2.19 Atividades

Questões conceituais

1. O que significa representar um algoritmo?
2. Cite três formas utilizadas para representar algoritmos.
3. Qual é a principal vantagem da linguagem natural?
4. Qual problema pode ocorrer quando utilizamos linguagem natural para representar algoritmos?
5. O que é um fluxograma?
6. Qual é a função das setas em um fluxograma?
7. Qual símbolo é utilizado normalmente para representar um processamento?
8. Qual símbolo representa entrada ou saída de dados?

9. Qual símbolo é utilizado para representar uma decisão?
10. O que é Português Estruturado?
11. Qual é a relação entre pseudocódigo e Portugol?
12. Por que o pseudocódigo pode facilitar o aprendizado inicial de programação?

Exercícios de identificação

Para cada situação a seguir, identifique **entrada**, **processamento** e **saída**.

1. Calcular a área de um retângulo conhecendo sua base e altura.
2. Calcular a idade aproximada de uma pessoa utilizando seu ano de nascimento.
3. Ler três notas e calcular a média.
4. Converter uma temperatura fornecida em graus Celsius para outra escala.
5. Ler o preço de um produto e calcular um novo preço após um acréscimo.

Construção de algoritmos

Represente inicialmente em linguagem natural e, posteriormente, utilizando um fluxograma:

1. Ler um número e apresentar seu dobro.
2. Ler dois números e apresentar sua soma.
3. Ler a base e a altura de um retângulo e calcular sua área.
4. Ler o nome de uma pessoa e apresentar uma mensagem de boas-vindas.
5. Ler três valores e apresentar sua soma.

Do fluxograma para o Portugol

Construa a representação em Portugol para os seguintes algoritmos:

1. cálculo do dobro de um número;
2. cálculo da soma de dois números;
3. cálculo da área de um retângulo;
4. cálculo da média de duas notas.

2.20 Desafio

Desafio – Três formas, uma única solução

Uma escola deseja um algoritmo simples capaz de receber duas notas de um estudante e calcular sua média aritmética. Realize as seguintes etapas:

1. identifique os dados de entrada;
2. identifique o processamento necessário;
3. identifique o dado de saída;
4. descreva a solução utilizando linguagem natural;
5. construa o fluxograma correspondente;
6. represente a mesma solução utilizando Portugol;
7. compare as três representações e identifique quais informações permanecem iguais entre elas.

Capítulo 3

Elementos Fundamentais dos Algoritmos

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender o papel dos dados em um algoritmo;
- diferenciar constantes e variáveis;
- compreender a relação entre variáveis e memória;
- reconhecer os principais tipos de dados utilizados no Portugol;
- criar identificadores adequados para variáveis;
- declarar variáveis utilizando a sintaxe do VisuAlg;
- utilizar corretamente o operador de atribuição;
- acompanhar as alterações de valores de uma variável;
- identificar erros comuns envolvendo tipos de dados e atribuições;
- construir algoritmos simples utilizando variáveis e constantes.

Os algoritmos trabalham constantemente com informações. Ao calcular a média de um estudante, por exemplo, precisamos armazenar suas notas. Ao calcular o preço final de um produto, precisamos representar seu preço. Ao registrar os dados de uma pessoa, podemos precisar armazenar seu nome, idade e outras informações.

Esses valores são chamados genericamente de **dados**. Durante a execução de um algoritmo, alguns dados permanecem fixos, enquanto outros podem mudar. Para representar essas situações utilizamos principalmente dois conceitos:

- **constantes**;
- **variáveis**.

Esses elementos formam uma das bases da programação de computadores.

Ideia central

Um algoritmo recebe, armazena, modifica e utiliza dados para produzir resultados.

3.1 Dados

Um **dado** representa um valor que pode ser utilizado durante a execução de um algoritmo. Exemplos de dados são:

- idade de uma pessoa;
- nome de um estudante;
- temperatura de uma sala;
- quantidade de produtos;
- preço de uma mercadoria;
- resultado de uma comparação;
- situação de aprovação ou reprovação.

Os dados podem possuir diferentes naturezas. O valor: 18 representa um número inteiro. Já: 8.5 representa um número que possui uma parte decimal. Enquanto: "Maria" representa um texto.

Também podemos ter valores lógicos, como: VERDADEIRO ou FALSO. Por esse motivo, precisamos conhecer os diferentes **tipos de dados** utilizados nos algoritmos.

3.2 Constantes

Uma **constante** representa um valor que permanece fixo durante a utilização de uma determinada expressão ou instrução. Exemplos: 10, 3.14, "Algoritmos", VERDADEIRO. No VisuAlg, podemos trabalhar principalmente com constantes:

- numéricas;
- de caracteres;
- lógicas.

3.2.1 Constantes numéricas

Representam valores numéricos. Podem ser inteiras: **10, 25, -4, 100**, ou reais: 7.5, 3.14, -10.25. No VisuAlg, o separador decimal utilizado é o **ponto**. Assim, escrevemos: 8.5 e não 8,5.

3.2.2 Constantes de caracteres

Uma constante de caracteres representa um texto. No VisuAlg, textos são escritos entre aspas duplas. Exemplos:

```
"Maria"  
"Curso Técnico em Informática"  
"Algoritmos"  
"A"  
"123"
```

Observe que: 123 representa um número. Enquanto: "123", representa um texto. Embora visualmente sejam semelhantes, esses valores possuem tipos diferentes.

3.2.3 Constantes lógicas

Constantes lógicas representam apenas duas possibilidades: **VERDADEIRO** ou **FALSO**. Esses valores serão particularmente importantes quando estudarmos operadores relacionais, operadores lógicos e estruturas condicionais.

3.3 Variáveis

Uma **variável** representa uma região da memória utilizada para armazenar temporariamente um valor durante a execução de um algoritmo. Seu conteúdo pode ser consultado ou alterado ao longo da execução.

Uma maneira simples de compreender esse conceito consiste em imaginar uma variável como uma **caixa identificada por um nome**. Por exemplo:

idade 16

Nesse exemplo:

- o nome da variável é **idade**;
- o valor armazenado atualmente é **16**.

Mais tarde, o valor dessa variável poderá ser alterado. Por exemplo:

```
idade <- 16  
idade <- 17
```

Após a segunda atribuição, o valor anterior deixa de ser o valor corrente da variável.

Importante

Uma variável pode assumir diferentes valores durante a execução de um algoritmo, mas armazena apenas um valor de cada vez.

3.4 Variáveis e Memória

Quando declaramos uma variável, estamos informando que o algoritmo precisará armazenar determinado tipo de dado. De maneira simplificada, podemos imaginar a memória como um conjunto de posições destinadas ao armazenamento de informações. Por exemplo:

nome "Ana"	idade 16	nota 8.5
----------------------	--------------------	--------------------

Cada variável possui:

- um **nome**;
- um **tipo**;
- um **valor**.

O nome permite localizar logicamente a informação no algoritmo.

O tipo determina quais valores podem ser armazenados.

O valor corresponde ao conteúdo atual da variável.

3.5 Tipos de Dados

O VisuAlg utiliza quatro tipos básicos de dados (**visualgRef**):

- inteiro;
- real;
- caractere;
- logico.

A Tabela [3.1](#) apresenta um resumo.

Tabela 3.1: Tipos básicos de dados utilizados no VisuAlg.

Tipo	Descrição	Exemplos
inteiro	Representa números sem parte decimal.	10, -5, 200
real	Representa números que podem possuir parte decimal.	7.5, 3.14, -2.8
caractere	Representa textos ou cadeias de caracteres.	"Ana", "IFPA"
logico	Representa valores lógicos.	VERDADEIRO, FALSO

3.5.1 Tipo inteiro

O tipo **inteiro** é utilizado quando precisamos armazenar números sem parte decimal. Exemplos de informações que normalmente podem ser representadas utilizando valores inteiros:

- idade;
- quantidade de alunos;
- quantidade de produtos;
- número de páginas;
- ano de nascimento.

Exemplo de declaração:

```
idade: inteiro
```

3.5.2 Tipo real

O tipo **real** é utilizado para armazenar números que podem possuir parte decimal. Exemplos:

- altura;
- peso;
- temperatura;
- média escolar;
- preço;
- distância.

Exemplo:

```
media: real
```

3.5.3 Tipo caractere

O tipo **caractere** é utilizado para representar textos. Apesar do nome, uma variável desse tipo pode armazenar uma cadeia de caracteres. Exemplos:

```
nome <- "Carlos"  
cidade <- "Braganca"  
curso <- "Tecnico em Informatica"
```

Neste material adotaremos o termo **caractere**, conforme a sintaxe utilizada pelo VisuAlg.

3.5.4 Tipo lógico

O tipo lógico representa uma informação que pode assumir somente:

VERDADEIRO

ou:

FALSO

Exemplos de informações que podem ser representadas utilizando valores lógicos:

- aluno aprovado;
- usuário autenticado;
- produto disponível;
- resposta correta;
- maior de idade.

Por exemplo:

```
aprovado <- VERDADEIRO
```

3.6 Identificadores (Nome)

O nome utilizado para representar uma variável é chamado de **identificador**. Um bom identificador deve ajudar o leitor a compreender qual informação está sendo armazenada. Compare:

X

com:

idadeAluno

Nos dois casos o nome pode representar uma variável válida, mas o segundo (idadeAluno) fornece mais informações sobre sua finalidade.

Boa prática

Sempre que possível, escolha nomes de variáveis que indiquem claramente o significado do dado armazenado.

3.7 Regras para Nomes de Variáveis

Os identificadores, nomes de variáveis, devem seguir determinadas regras. De maneira geral:

- devem começar por uma letra;
- podem conter letras, números e o caractere _;
- não devem possuir espaços;
- não devem utilizar palavras reservadas da linguagem;
- duas variáveis diferentes não devem possuir o mesmo nome.

Exemplos de identificadores adequados:

```
idade
nota1
nota2
media
nome_aluno
quantidadeProdutos
preco_final
```

Exemplos inadequados:

```
2nota
nome aluno
media-final
para
inicio
```

3.8 Palavras Reservadas

Algumas palavras possuem significado especial na linguagem e, por isso, não devem ser utilizadas como nomes de variáveis. Exemplos:

```
algoritmo
inicio
finalgoritmo
var
inteiro
real
logico
se
senao
para
enquanto
leia
escreva
```

Essas palavras são chamadas de **palavras reservadas**. Por exemplo, não devemos escrever:

```
var
  inicio: inteiro
```

pois `inicio` já possui uma função específica na sintaxe do algoritmo.

3.9 Declaração de Variáveis

Antes de utilizar uma variável em um algoritmo, devemos informar seu nome e seu tipo. No VisuAlg, as variáveis são declaradas na seção iniciada pela palavra:

```
var
```

A forma geral é:

```
nome_da_variavel: tipo
```

Por exemplo:

```
var
  idade: inteiro
  altura: real
  nome: caractere
  aprovado: logico
```

Também é possível declarar várias variáveis do mesmo tipo na mesma linha. Por exemplo:

```
var
  nota1, nota2, media: real
```

Nesse caso, as três variáveis possuem o tipo `real`.

3.10 Exemplo de Declaração

Considere um algoritmo que precise trabalhar com os seguintes dados de um estudante:

- nome;
- idade;
- duas notas;
- média;
- situação de aprovação.

Uma possível declaração seria:

```
var
  nome: caractere
  idade: inteiro
  nota1, nota2, media: real
  aprovado: logico
```

Observe que cada dado foi associado ao tipo mais adequado.

3.11 Atribuição

Depois de declarar uma variável, podemos armazenar valores nela. Essa operação é chamada de **atribuição**. No VisuAlg, o operador de atribuição é:

<-

Podemos interpretar esse símbolo como a palavra:

recebe

Assim:

```
idade <- 16
```

pode ser lido como: "idade recebe 16". Outro exemplo:

```
nome <- "Mariana"
```

pode ser lido como: "nome recebe Mariana".

3.12 Atribuição Não é Igualdade Matemática

É importante não confundir o operador de atribuição com uma igualdade matemática. Considere:

```
contador <- contador + 1
```

Na Matemática, uma expressão semelhante:

$$\text{contador} = \text{contador} + 1$$

não faria sentido como igualdade. Em programação, entretanto, a instrução significa:

1. obter o valor atual de **contador**;
2. adicionar 1;
3. armazenar o novo valor novamente em **contador**.

Se inicialmente:

```
contador <- 5
```

e depois executarmos:

```
contador <- contador + 1
```

o novo valor será: **6**

Atenção

Em Portugal, <- representa **atribuição**. O símbolo = será utilizado posteriormente em comparações.

3.13 Atribuindo o Valor de Outra Variável

Uma variável também pode receber o conteúdo de outra variável. Considere:

```
a <- 10
```

```
b <- a
```

Após essas instruções:

```
a = 10
```

e:

```
b = 10
```

Entretanto, **a** e **b** continuam sendo variáveis diferentes. **Se depois executarmos:**

```
a <- 20
```

teremos:

```
a = 20
```

e:

```
b = 10
```

Modificar uma variável não altera automaticamente outra variável que anteriormente recebeu seu valor.

3.14 Atribuindo o Resultado de uma Expressão

Uma variável também pode receber o resultado de uma expressão. Por exemplo:

```
a <- 10
b <- 5
soma <- a + b
```

A variável `soma` receberá:

15

Outro exemplo:

```
nota1 <- 8.0
nota2 <- 6.0

media <- (nota1 + nota2) / 2
```

Nesse caso:

$$media = \frac{8 + 6}{2}$$

portanto:

$$media = 7$$

As expressões aritméticas serão estudadas detalhadamente no próximo capítulo.

3.15 Compatibilidade de Tipos

O valor atribuído a uma variável deve ser compatível com o tipo declarado. Considere:

```
var
  idade: inteiro
```

A atribuição:

```
idade <- 17
```

é adequada. Entretanto:

```
idade <- "dezessete"
```

não representa um valor compatível com uma variável do tipo `inteiro`.

Outro exemplo:

```
var
  nome: caractere
```

A atribuição:

```
nome <- "Maria"
```

é compatível com o tipo declarado.

3.16 Erros Comuns

Durante o aprendizado inicial, alguns erros aparecem com frequência.

3.16.1 Utilizar uma variável sem declará-la

Exemplo inadequado:

```
algoritmo "exemplo"
```

```
inicio
```

```
    idade <- 16
```

```
fimalgoritmo
```

A variável `idade` deveria ser declarada antes de ser utilizada.

3.16.2 Utilizar um nome inválido

Exemplo:

```
2nota: real
```

Um identificador não deve começar por número.

3.16.3 Utilizar espaço no identificador (nome variável)

Exemplo:

```
nome aluno: caractere
```

Uma opção adequada seria:

```
nome_aluno: caractere
```

3.16.4 Utilizar uma palavra reservada

Exemplo:

```
inicio: inteiro
```

Nesse caso, `inicio` já possui um significado especial no Portugol.

3.16.5 Atribuir um valor incompatível

Exemplo:

```
var
    quantidade: inteiro

inicio

    quantidade <- "dez"
```

fimalgoritmo

O texto "dez" não representa um valor inteiro.

3.17 Exemplo Completo

Considere um algoritmo simples que representa informações de um estudante.

```
algoritmo "dados_estudante"

var
    nome: caractere
    idade: inteiro
    nota1, nota2, media: real
    aprovado: logico

inicio

    nome <- "Ana"
    idade <- 16

    nota1 <- 8.0
    nota2 <- 7.0

    media <- (nota1 + nota2) / 2

    aprovado <- VERDADEIRO

    escreval("Nome: ", nome)
    escreval("Idade: ", idade)
    escreval("Media: ", media)
    escreval("Aprovado: ", aprovado)

fimalgoritmo
```

Esse exemplo reúne os principais elementos estudados neste capítulo:

- declaração de variáveis;
- diferentes tipos de dados;
- constantes;
- atribuições;
- utilização de expressões;
- atualização e leitura dos valores armazenados.

O comando **escreval** já foi apresentado anteriormente e será estudado com mais detalhes no capítulo dedicado à entrada e saída de dados.

3.18 Resumo do Capítulo

Neste capítulo estudamos alguns dos elementos fundamentais utilizados na construção de algoritmos. Os **dados** representam as informações manipuladas pelo algoritmo.

Uma **constante** representa um valor fixo, enquanto uma **variável** corresponde a uma região identificada da memória cujo conteúdo pode mudar durante a execução.

No VisuAlg, os quatro tipos básicos utilizados nesta apostila são:

- inteiro;
- real;
- caractere;
- logico.

O nome de uma variável é chamado de **identificador** e deve seguir as regras da linguagem. As variáveis são declaradas na seção iniciada por **var**.

A atribuição de valores é realizada utilizando o operador:

<-

que pode ser lido como “recebe”. Também vimos que uma variável pode receber constantes, valores de outras variáveis ou resultados de expressões.

3.19 Atividades

Questões conceituais

1. O que é um dado?
2. O que é uma constante?
3. O que é uma variável?
4. Qual é a relação entre uma variável e a memória do computador?

5. Quais são os quatro tipos básicos de dados utilizados no VisuAlg?
6. Qual tipo seria adequado para armazenar a idade de uma pessoa?
7. Qual tipo seria adequado para armazenar a altura de uma pessoa?
8. Qual tipo seria adequado para armazenar o nome de um estudante?
9. Qual tipo seria adequado para representar uma informação que pode ser verdadeira ou falsa?
10. O que é um identificador?
11. Qual é a finalidade da declaração de uma variável?
12. Qual operador representa atribuição no VisuAlg?
13. Qual é a diferença entre atribuição e igualdade matemática?

Classificação de Dados

Indique o tipo mais adequado para representar cada informação:

1. idade de uma pessoa;
2. nome de uma cidade;
3. temperatura;
4. quantidade de alunos;
5. salário;
6. nome de um estudante;
7. aluno aprovado ou não;
8. número de páginas de um livro;
9. altura de uma pessoa;
10. resposta correta ou incorreta.

Constante ou Variável?

Classifique cada elemento abaixo como uma possível constante ou variável, considerando um algoritmo que calcula a média de duas notas:

1. 2;
2. nota1;
3. nota2;
4. media;
5. 7.0;
6. "Aprovado".

Identificadores

Indique quais identificadores a seguir são adequados para utilização como nomes de variáveis no padrão adotado nesta apostila.

1. idade
2. nota1
3. 1nota
4. nome_aluno
5. nome aluno
6. mediaFinal
7. preco-final
8. inicio
9. quantidade
10. valor2

Declaração de Variáveis

Escreva a declaração adequada para representar:

1. nome e idade de uma pessoa;
2. três notas e uma média;
3. preço e quantidade de um produto;
4. nome de um aluno e sua situação de aprovação;
5. largura, altura e área de um retângulo.

Atribuições

Considerando as declarações:

```
var
```

```
    a, b, c: inteiro
```

```
    determine os valores finais de a, b e c:
```

```
a <- 5
```

```
b <- 10
```

```
c <- a + b
```

```
a <- 20
```

```
b <- a
```

3.20 Desafio

Desafio – Acompanhe a memória

Considere o seguinte algoritmo:

```
algoritmo "memoria"

var
    a, b, c: inteiro

inicio

    a <- 5
    b <- 2

    c <- a
    a <- b
    b <- c

finalgoritmo
```

Sem executar o algoritmo no computador:

1. determine o valor inicial de cada variável após as duas primeiras atribuições;
2. construa um teste de mesa completo;
3. determine os valores finais de **a**, **b** e **c**;
4. explique, com suas próprias palavras, o que ocorreu com os valores originalmente armazenados em **a** e **b**.

Capítulo 4

Operadores e Expressões

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender o conceito de expressão;
- reconhecer operadores e operandos;
- utilizar corretamente operadores aritméticos;
- compreender a precedência das operações aritméticas;
- transformar expressões matemáticas em expressões computacionais;
- utilizar operadores relacionais;
- construir expressões que resultem em valores lógicos;
- compreender os operadores lógicos `e`, `ou`, `nao` e `xou`;
- interpretar tabelas-verdade;
- utilizar parênteses para organizar expressões;
- construir expressões literais;
- utilizar concatenação de cadeias de caracteres;
- avaliar expressões passo a passo.

No capítulo anterior estudamos variáveis, constantes e tipos de dados. Entretanto, apenas armazenar informações não é suficiente para resolver a maioria dos problemas computacionais. Precisamos também realizar operações sobre esses dados. Considere os seguintes exemplos:

- somar duas notas;
- calcular a média de um estudante;
- verificar se uma idade é maior ou igual a 18;

- determinar se uma nota está entre 0 e 10;
- verificar se duas condições são verdadeiras ao mesmo tempo;
- combinar diferentes textos para formar uma mensagem.

Para realizar essas tarefas utilizamos **operadores** e **expressões**.

Ideia central

Uma **expressão** combina valores, variáveis e operadores para produzir um novo valor.

4.1 Operadores e Operandos

Um **operador** representa uma operação que será realizada sobre um ou mais valores. Os valores utilizados por um operador são chamados de **operandos**. Considere:

$5 + 3$

Nesse exemplo:

- $+$ é o operador;
- 5 é um operando;
- 3 é outro operando.

O resultado da expressão é:

$$5 + 3 = 8$$

Os operandos também podem ser variáveis.

Por exemplo:

`nota1 + nota2`

Se:

`nota1 <- 8`

`nota2 <- 6`

então:

$$nota1 + nota2 = 14$$

4.2 O que é uma Expressão?

Uma **expressão** é uma combinação de constantes, variáveis, operadores e, quando necessário, parênteses. Exemplos:

`10 + 5`

`nota1 + nota2`

`(base * altura)`

`(idade >= 18)`

`(media >= 7) e (frequencia >= 75)`

Cada expressão produz um resultado. Esse resultado pode ser:

- numérico;
- lógico;
- caractere.

Podemos, portanto, trabalhar com diferentes tipos de expressões. Neste capítulo estudaremos principalmente:

- expressões aritméticas;
- expressões relacionais;
- expressões lógicas;
- expressões literais.

4.3 Expressões Aritméticas

As **expressões aritméticas** são utilizadas para realizar cálculos numéricos. Podemos utilizar constantes:

`5 + 3`

variáveis:

`nota1 + nota2`

ou uma combinação entre elas:

`salario + 200`

O resultado de uma expressão aritmética também pode ser armazenado em uma variável. Por exemplo:

`soma <- a + b`

ou:

`media <- (nota1 + nota2) / 2`

4.4 Operadores Aritméticos

O VisuAlg disponibiliza operadores para as principais operações aritméticas **visualgRef**.

Tabela 4.1: Principais operadores aritméticos utilizados no VisuAlg.

Operador	Operação	Exemplo
+	Adição	5 + 3 resulta em 8.
-	Subtração	10 - 4 resulta em 6.
*	Multiplicação	5 * 3 resulta em 15.
/	Divisão	10 / 4 resulta em 2.5.
\	Divisão inteira	5 \2 resulta em 2.
%	Resto da divisão	8 % 3 resulta em 2.
^	Potenciação	5 ^ 2 resulta em 25.

4.5 Adição e Subtração

A adição é representada pelo operador:

+

Por exemplo:

```
soma <- 10 + 5
```

faz com que **soma** receba o valor: **15**. A subtração é representada por:

-

Por exemplo:

```
diferenca <- 20 - 8
```

resulta em: **12**

4.6 Multiplicação

A multiplicação é representada pelo símbolo:

*

Em algoritmos, normalmente não utilizamos o símbolo matemático:

×

nem escrevemos:

5×3

Para multiplicar 5 por 3 utilizamos: **5 * 3**. Exemplo:

```
area <- base * altura
```

4.7 Divisão

A divisão comum é representada pelo operador:

/

Por exemplo:

```
media <- soma / quantidade
```

Se:

```
soma <- 15  
quantidade <- 2
```

então:

$$media = 7.5$$

4.8 Divisão Inteira

O VisuAlg também possui um operador específico para a **divisão inteira**. Esse operador é representado por:

\

Por exemplo:

```
5 \ 2
```

resulta em:

```
2
```

A parte fracionária da divisão não é considerada no resultado da divisão inteira.

4.9 Resto da Divisão

O operador:

%

retorna o resto de uma divisão inteira. Por exemplo:

$$8 \div 3 = 2$$

com resto:

2

Assim:

`8 % 3`

resulta em:

`2`

Esse operador será bastante útil posteriormente. Por exemplo, podemos utilizá-lo para verificar se um número é divisível por outro. Se:

`numero % 2`

resultar em zero, então o número é divisível por 2.

4.10 Potenciação

No VisuAlg, a potenciação pode ser representada pelo operador:

`^` (acento circunflexo, chapeuzinho)

Por exemplo:

`5 ^ 2`

representa:

$$5^2$$

e resulta em: **25**. Outro exemplo:

`quadrado <- numero ^ 2`

4.11 Linearização de Expressões

Em um algoritmo, as expressões normalmente são escritas em uma única linha. Esse processo é chamado de **linearização de expressões** (pereira2011visualg). Considere:

$$\frac{a + b}{2}$$

Em Portugol escrevemos:

`(a + b) / 2`

Outro exemplo:

$$\frac{a + b}{c + d}$$

deve ser escrito como:

`(a + b) / (c + d)`

Atenção

Ao transformar uma expressão matemática em uma expressão computacional, os parênteses são fundamentais para preservar a ordem correta das operações.

4.12 Precedência dos Operadores Aritméticos

Quando uma expressão possui vários operadores, existe uma ordem para a execução das operações. De maneira simplificada, podemos considerar a seguinte ordem:

1. expressões entre parênteses;
2. potenciação;
3. multiplicação, divisão, divisão inteira e resto;
4. adição e subtração.

Assim:

$$2 + 3 * 4$$

não significa:

$$(2 + 3) \times 4$$

A multiplicação é realizada primeiro:

$$2 + (3 \times 4)$$

portanto:

$$2 + 12 = 14$$

4.13 Utilizando Parênteses

Os parênteses podem alterar a ordem de avaliação de uma expressão. Compare:

$$2 + 3 * 4$$

com:

$$(2 + 3) * 4$$

Na primeira expressão:

$$2 + 3 \times 4 = 14$$

Na segunda:

$$(2 + 3) \times 4 = 20$$

Portanto, os parênteses podem modificar completamente o resultado.

4.14 Exemplo Resolvido: Média

Considere:

```
nota1 <- 8  
nota2 <- 6
```

```
media <- (nota1 + nota2) / 2
```

Primeiro avaliamos:

$$nota1 + nota2$$

substituindo os valores:

$$8 + 6 = 14$$

Depois:

$$14/2 = 7$$

Assim:

```
media <- 7
```

4.15 Operadores Relacionais

Nem sempre desejamos realizar apenas cálculos. Muitas vezes precisamos **comparar valores**. Por exemplo:

- a idade é maior ou igual a 18?
- a média é maior ou igual a 7?
- dois valores são iguais?
- um número é diferente de outro?

Para realizar essas comparações utilizamos os **operadores relacionais**. O resultado de uma comparação é sempre um valor lógico:

VERDADEIRO

ou:

FALSO

4.16 Principais Operadores Relacionais

Tabela 4.2: Operadores relacionais utilizados no VisuAlg.

Operador	Significado	Exemplo
=	Igual	5 = 5
<>	Diferente	5 <> 3
>	Maior que	10 > 5
<	Menor que	3 < 8
>=	Maior ou igual	idade >= 18
<=	Menor ou igual	nota <= 10

4.17 Resultados das Expressões Relacionais

Considere:

5 > 3

O resultado é:

VERDADEIRO

Já:

5 < 3

resulta em:

FALSO

Outro exemplo:

idade <= 17

A expressão:

idade >= 18

resulta em:

FALSO

4.18 Atribuição e Comparação

É importante distinguir dois operadores que possuem funções diferentes. A atribuição:

```
idade <- 18
```

significa: **idade recebe 18**. A expressão: **idade = 18**, representa uma comparação: **idade é igual a 18?**

Não confunda

<- atribuição

= comparação

4.19 Expressões Lógicas

Uma **expressão lógica** produz como resultado:

VERDADEIRO

ou:

FALSO

Expressões relacionais já produzem resultados lógicos. Por exemplo:

```
idade >= 18
```

Entretanto, frequentemente precisamos combinar duas ou mais condições. Por exemplo: O estudante possui média maior ou igual a 7 e frequência maior ou igual a 75?

Para combinar condições utilizamos operadores lógicos.

4.20 Operadores Lógicos

O VisuAlg utiliza principalmente os seguintes operadores lógicos:

Tabela 4.3: Operadores lógicos utilizados no VisuAlg.

Operador	Operação	Descrição
e	Conjunção	Resulta em VERDADEIRO quando as duas condições são verdadeiras.
ou	Disjunção	Resulta em VERDADEIRO quando pelo menos uma das condições é verdadeira.
nao	Negação	Inverte o valor lógico de uma expressão.
xou	Ou exclusivo	Resulta em VERDADEIRO quando os dois valores lógicos são diferentes.

4.21 Operador e

O operador: **e**, exige que as duas condições sejam verdadeiras para que o resultado final seja VERDADEIRO.

Considere:

`(media >= 7) e (frequencia >= 75)`

Para que a expressão seja verdadeira:

- a média deve ser maior ou igual a 7;
- a frequência deve ser maior ou igual a 75.

Se uma das condições for falsa, o resultado será FALSO.

4.22 Tabela-Verdade do Operador e

Tabela 4.4: Tabela-verdade do operador e.

A	B	A e B
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	FALSO
FALSO	VERDADEIRO	FALSO
FALSO	FALSO	FALSO

4.23 Operador ou

O operador: **ou**, produz VERDADEIRO quando pelo menos uma das condições é verdadeira. Por exemplo:

`(dia = "sabado") ou (dia = "domingo")`

A expressão será verdadeira se o valor de `dia` for:

- "sabado";
- ou "domingo".

4.24 Tabela-Verdade do Operador ou

Tabela 4.5: Tabela-verdade do operador ou.

A	B	A ou B
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

4.25 Operador nao

O operador: **nao**, inverte o valor lógico de uma expressão. Assim: **nao VERDADEIRO** resulta em:

FALSO

e:

nao FALSO

resulta em:

VERDADEIRO

4.26 Tabela-Verdade do Operador nao

Tabela 4.6: Tabela-verdade do operador nao.

A	nao A
VERDADEIRO	FALSO
FALSO	VERDADEIRO

4.27 Operador xou

O operador: **xou**, representa o chamado **ou exclusivo**. Seu resultado é VERDADEIRO quando os dois valores lógicos são diferentes.

Tabela 4.7: Tabela-verdade do operador `xou`.

A	B	A xou B
VERDADEIRO	VERDADEIRO	FALSO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

4.28 Combinando Expressões Lógicas

Considere:

```
idade <- 17
autorizacao <- VERDADEIRO
```

Podemos construir:

```
(idade >= 18) ou autorizacao
```

Primeiro:

```
idade >= 18
```

resulta em:

```
FALSO
```

Mas:

```
autorizacao
```

é:

```
VERDADEIRO
```

Portanto:

FALSO ou VERDADEIRO

resulta em:

```
VERDADEIRO
```

4.29 Uso de Parênteses em Expressões Lógicas

Em expressões que combinam várias condições, utilizaremos parênteses para tornar a ordem de avaliação explícita. Por exemplo:

```
(media >= 7) e (frequencia >= 75)
```

ou:

```
(idade >= 18) ou (autorizacao = VERDADEIRO)
```

Essa forma torna a expressão mais legível e reduz possíveis ambiguidades.

Boa prática

Ao combinar comparações com operadores lógicos, utilize parênteses para deixar claramente identificada cada condição.

4.30 Expressões Literais

Expressões também podem envolver valores do tipo **caractere**. No VisuAlg, o operador: **+**, pode ser utilizado para realizar a **concatenação** de cadeias de caracteres. Concatenar significa unir textos.

4.31 Concatenação

Considere:

```
"Bom" + " dia"
```

O resultado será:

```
"Bom dia"
```

Outro exemplo:

```
nome <- "Ana"
```

```
mensagem <- "Ola, " + nome
```

A variável **mensagem** receberá:

```
"Ola, Ana"
```

Observe que os espaços também fazem parte das cadeias de caracteres.

4.32 Comparação entre Caracteres

Valores do tipo caractere também podem participar de comparações. Por exemplo:

```
nome = "Ana"
```

produzirá um resultado lógico. Se:

```
nome <- "Ana"
```

então:

```
nome = "Ana"
```

resultará em:

VERDADEIRO

4.33 Expressões Compostas

Em algoritmos reais, frequentemente combinamos diferentes tipos de operadores. Considere:

```
(media >= 7) e (idade >= 16)
```

Nessa expressão existem:

- duas comparações relacionais;
- um operador lógico.

Outro exemplo:

```
((nota1 + nota2) / 2) >= 7
```

Nesse caso temos:

- operações aritméticas;
- uma operação relacional.

O resultado final será lógico (VERDADEIRO ou FALSO).

4.34 Avaliando uma Expressão Passo a Passo

Considere:

```
nota1 <- 8  
nota2 <- 6
```

```
((nota1 + nota2) / 2) >= 7
```

Primeiro:

$$nota1 + nota2$$
$$8 + 6 = 14$$

Depois:

$$14/2 = 7$$

Finalmente:

$$7 \geq 7$$

Logo, o resultado é:

VERDADEIRO

4.35 Outro Exemplo Resolvido

Considere:

```
idade <- 17  
nota <- 8
```

```
(idade >= 16) e (nota >= 7)
```

Primeira condição:

$$17 \geq 16$$

resultado:

VERDADEIRO

Segunda condição:

$$8 \geq 7$$

resultado:

VERDADEIRO

Portanto:

VERDADEIRO e VERDADEIRO

resulta em:

VERDADEIRO

4.36 Erros Comuns

4.36.1 Utilizar x para multiplicação

Inadequado:

```
area <- base x altura
```

Adequado:

```
area <- base * altura
```

4.36.2 Confundir atribuição com comparação

Atribuição:

```
idade <- 18
```

Comparação:

```
idade = 18
```

4.36.3 Esquecer os parênteses

Considere a média:

```
media <- nota1 + nota2 / 2
```

Essa expressão não representa corretamente a média aritmética das duas notas, pois a divisão será realizada antes da soma. A forma adequada é:

```
media <- (nota1 + nota2) / 2
```

4.36.4 Esquecer aspas em textos

Inadequado:

```
nome <- Ana
```

Adequado:

```
nome <- "Ana"
```

4.37 Resumo do Capítulo

Neste capítulo estudamos operadores e expressões. Uma expressão combina valores, variáveis e operadores para produzir um resultado. Os principais operadores aritméticos estudados foram:

+ - * / \ % ^

Também vimos que a ordem das operações influencia o resultado e que os parênteses podem ser utilizados para definir explicitamente essa ordem. Os operadores relacionais:

= <> < > <= >=

permitem comparar valores e produzem resultados lógicos. Os principais operadores lógicos estudados foram:

e ou nao xou

Finalmente, vimos que valores do tipo caractere também podem ser combinados, principalmente por meio da concatenação.

4.38 Atividades

Questões conceituais

1. O que é um operador?
2. O que é um operando?
3. O que é uma expressão?
4. Qual operador representa multiplicação no Portugal?
5. Qual é a diferença entre / e \?
6. Para que serve o operador %?
7. Qual é a função dos parênteses em uma expressão?
8. O que é um operador relacional?
9. Qual tipo de resultado é produzido por uma expressão relacional?
10. Qual é a diferença entre <- e =?
11. Qual é a função do operador lógico e?
12. Qual é a função do operador lógico ou?
13. Qual é a função do operador nao?
14. O que significa concatenar textos?

Calcule o Resultado

Determine o resultado de cada expressão:

1. $5 + 3$
2. $10 - 4$
3. $3 * 5$
4. $10 / 4$
5. $10 \setminus 4$
6. $10 \% 4$
7. $2 ^ 3$
8. $5 + 2 * 3$
9. $(5 + 2) * 3$
10. $20 / 2 + 5$
11. $20 / (2 + 3)$

Expressões Relacionais

Indique se cada expressão resulta em **VERDADEIRO** ou **FALSO**.

1. $5 > 3$
2. $5 < 3$
3. $10 = 10$
4. $10 <> 10$
5. $7 >= 7$
6. $9 <= 5$
7. $3 + 2 = 5$
8. $10 / 2 > 4$

Expressões Lógicas

Considere:

```
a <- 5  
b <- 8  
c <- 5
```

Determine o resultado de:

1. $a = c$
2. $a > b$
3. $(a = c)$ e $(b > a)$
4. $(a > b)$ ou $(c = a)$
5. `nao` $(a = c)$
6. $(a < b)$ xou $(a = c)$

Construa as Expressões

Escreva uma expressão em Portugol para representar cada situação.

1. a idade é maior ou igual a 18;
2. a nota está entre 0 e 10;
3. a média é maior ou igual a 7;
4. a média é maior ou igual a 7 e a frequência é maior ou igual a 75;
5. o dia é sábado ou domingo;
6. o número é diferente de zero;
7. o valor está entre 10 e 20;
8. o usuário possui idade maior ou igual a 18 ou possui autorização.

Linearização

Transforme as expressões matemáticas em expressões adequadas para Portugol.

1.
$$\frac{a + b}{2}$$
2.
$$\frac{a + b + c}{3}$$

3.

$$\frac{x + y}{z}$$

4.

$$\frac{a + b}{c + d}$$

5.

$$\frac{(a + b) \times c}{d}$$

4.39 Desafio

Desafio – Descobrindo a Expressão

Uma escola considera que um estudante atende determinado critério quando:

- sua média é maior ou igual a 7;
- sua frequência é maior ou igual a 75;
- sua idade é maior ou igual a 14.

Considere:

```
media <- 8.0  
frequencia <- 80  
idade <- 15
```

Responda:

1. construa uma única expressão lógica que represente os três critérios;
2. avalie separadamente cada comparação;
3. determine o resultado lógico final;
4. altere a frequência para 70 e determine novamente o resultado;
5. explique por que o resultado mudou.

Entrada, Processamento e Saída de Dados

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender o modelo Entrada–Processamento–Saída;
- identificar os dados de entrada de um problema;
- identificar o processamento necessário;
- identificar os resultados que devem ser apresentados;
- utilizar o comando `leia` para receber dados;
- utilizar os comandos `escreva` e `escreval`;
- diferenciar entrada, processamento e saída;
- construir algoritmos sequenciais completos;
- utilizar mensagens para orientar a entrada de dados;
- realizar testes de mesa;
- identificar erros comuns em algoritmos sequenciais.

Até este momento estudamos os principais elementos utilizados para armazenar e manipular informações em um algoritmo. Vimos que variáveis podem armazenar valores e que expressões permitem realizar cálculos e comparações.

Entretanto, muitos algoritmos precisam também receber informações do usuário e apresentar resultados. Considere um programa destinado a calcular a média de um estudante. Para realizar essa tarefa, será necessário:

1. receber as notas;
2. calcular a média;
3. apresentar o resultado.

Essas três etapas correspondem ao modelo:

Entrada $\longrightarrow$ **Processamento** $\longrightarrow$ **Saída**

Esse modelo será utilizado diversas vezes ao longo da apostila.

5.1 O Modelo Entrada–Processamento–Saída

Uma forma simples de analisar um problema computacional consiste em identificar três componentes:

Entrada: informações fornecidas ao algoritmo;

Processamento: operações realizadas sobre essas informações;

Saída: resultados produzidos pelo algoritmo.

Considere o problema:

Ler dois números e apresentar sua soma.

Podemos identificar:

Entrada: dois números;

Processamento: somar os números;

Saída: resultado da soma.

5.2 Representação Gráfica

A Figura 5.1 apresenta uma representação simplificada do modelo Entrada–Processamento–Saída.

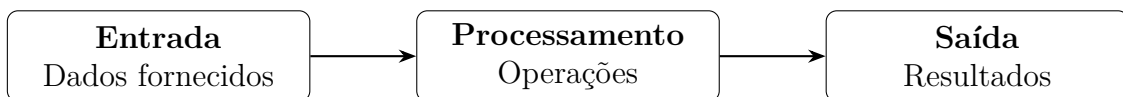


Figura 5.1: Modelo Entrada–Processamento–Saída.

Ideia central

Antes de escrever um algoritmo, pergunte:

1. Quais dados preciso receber?
2. O que preciso fazer com esses dados?
3. Qual resultado preciso apresentar?

5.3 Entrada de Dados

A **entrada de dados** corresponde às informações fornecidas ao algoritmo durante sua execução. Esses dados podem representar, por exemplo:

- nome;
- idade;
- nota;
- quantidade;
- preço;
- temperatura;
- altura.

No VisuAlg, utilizamos o comando:

```
leia(...)
```

para receber dados digitados pelo usuário **visualgRef**.

5.4 O Comando **leia**

A forma básica é:

```
leia(variavel)
```

Por exemplo:

```
leia(idade)
```

Quando essa instrução for executada, o VisuAlg aguardará que o usuário informe um valor. O valor digitado será armazenado na variável **idade**.

5.5 Exemplo de Entrada

Considere:

```
algoritmo "ler_idade"
```

```
var
```

```
    idade: inteiro
```

```
inicio
```

```
    leia(idade)
```

```
fimalgoritmo
```

Se o usuário digitar:

```
16
```

então a variável:

```
idade
```

passará a armazenar o valor:

```
16
```

5.6 Orientando o Usuário

Embora seja possível utilizar apenas o comando `leia`, é recomendável apresentar uma mensagem antes da leitura. Compare:

```
leia(idade)
```

com:

```
escreva("Digite sua idade: ")  
leia(idade)
```

No segundo caso, o usuário sabe claramente qual informação deve digitar.

Boa prática

Sempre que possível, apresente uma mensagem indicando qual informação deve ser fornecida antes de utilizar o comando `leia`.

5.7 Lendo Vários Dados

O comando `leia` também pode receber mais de uma variável. Por exemplo:

```
leia(nota1, nota2)
```

Entretanto, durante o aprendizado inicial, muitas vezes é mais claro realizar cada leitura separadamente:

```
escreva("Digite a primeira nota: ")  
leia(nota1)
```

```
escreva("Digite a segunda nota: ")  
leia(nota2)
```

Essa forma torna a interação com o usuário mais compreensível.

5.8 Processamento

O **processamento** corresponde às operações realizadas pelo algoritmo sobre os dados.

O processamento pode envolver:

- cálculos;
- atribuições;
- comparações;
- atualizações de variáveis;
- conversões;
- outras operações necessárias à solução do problema.

Considere:

```
media <- (nota1 + nota2) / 2
```

Nesse caso:

- `nota1` e `nota2` representam dados;
- a soma e a divisão representam o processamento;
- o resultado é armazenado na variável `media`.

5.9 Exemplo de Processamento

Considere:

```
base <- 5  
altura <- 4
```

```
area <- base * altura
```

O processamento corresponde a:

$$area = base \times altura$$

Substituindo os valores:

$$area = 5 \times 4$$

portanto:

$$area = 20$$

5.10 Saída de Dados

A **saída de dados** corresponde às informações produzidas pelo algoritmo e apresentadas ao usuário. No VisuAlg, os principais comandos de saída são:

`escreva(...)`

 e:

`escreval(...)`

`visualgRef`

5.11 O Comando `escreva`

O comando:

`escreva(...)`

 apresenta informações na saída sem avançar automaticamente para uma nova linha.
 Por exemplo:

`escreva("Algoritmos")`

 produz:

Algoritmos

5.12 O Comando `escreval`

O comando:

`escreval(...)`

 também apresenta informações, mas após a exibição avança para uma nova linha. Por exemplo:

`escreval("Algoritmos")`
`escreval("Programacao")`

 produz:

Algoritmos
Programacao

5.13 escreva e escreval

Considere:

```
escreva("A")  
escreva("B")  
escreva("C")
```

O resultado será semelhante a:

ABC

Agora considere:

```
escreval("A")  
escreval("B")  
escreval("C")
```

O resultado será:

A
B
C

Tabela 5.1: Diferença básica entre `escreva` e `escreval`.

Comando	Comportamento
<code>escreva</code>	Apresenta a informação e mantém a próxima saída na mesma linha.
<code>escreval</code>	Apresenta a informação e avança para uma nova linha.

5.14 Apresentando o Valor de uma Variável

Para apresentar o conteúdo de uma variável, escrevemos seu nome sem aspas. Por exemplo:

```
idade <- 16
```

```
escreval(idade)
```

A saída será:

16

Se utilizarmos:

```
escreval("idade")
```

a saída será:

idade

porque o texto está entre aspas.

5.15 Textos e Variáveis na Mesma Saída

Podemos combinar textos e variáveis. Por exemplo:

```
nome <- "Ana"
idade <- 16

escreval("Nome: ", nome)
escreval("Idade: ", idade)
```

A saída será semelhante a:

```
Nome: Ana
Idade: 16
```

Essa forma torna os resultados mais fáceis de compreender.

5.16 Apresentando Expressões

Os comandos de saída também podem apresentar o resultado de expressões. Por exemplo:

```
a <- 5
b <- 3

escreval(a + b)
```

A saída será:

```
8
```

Também podemos escrever:

```
escreval("Soma = ", a + b)
```

produzindo:

```
Soma = 8
```

5.17 Formatação Básica de Valores

O `VisuAlg` permite controlar a largura utilizada na apresentação de números e, no caso de valores reais, a quantidade de casas decimais (`visualgRef`). Por exemplo:

```
escreval(valor:6:2)
```

indica que o valor deverá ser apresentado utilizando espaço total de seis posições e duas casas decimais. Considere:

```
valor <- 8.5
```

```
escreval(valor:6:2)
```

A saída mostrará o número com duas casas decimais. Esse recurso é útil principalmente para valores como preços, médias e medidas.

5.18 Algoritmos Sequenciais

Um **algoritmo sequencial** executa suas instruções na ordem em que elas aparecem. Nesse tipo de algoritmo, não há ainda decisões nem repetições. O fluxo segue normalmente:

Entrada → Processamento → Saída

Esse tipo de algoritmo constitui a base para estruturas mais complexas.

5.19 Exemplo 1 – Soma de Dois Números

Problema

Ler dois números e apresentar sua soma.

Análise

Entrada: dois números;

Processamento: somar os dois valores;

Saída: soma.

Algoritmo

```
algoritmo "soma_dois_numeros"

var
    numero1, numero2, soma: real

inicio

    escreva("Digite o primeiro numero: ")
    leia(numero1)

    escreva("Digite o segundo numero: ")
    leia(numero2)

    soma <- numero1 + numero2

    escreval("Soma = ", soma)

fimalgoritmo
```

5.20 Fluxograma do Exemplo

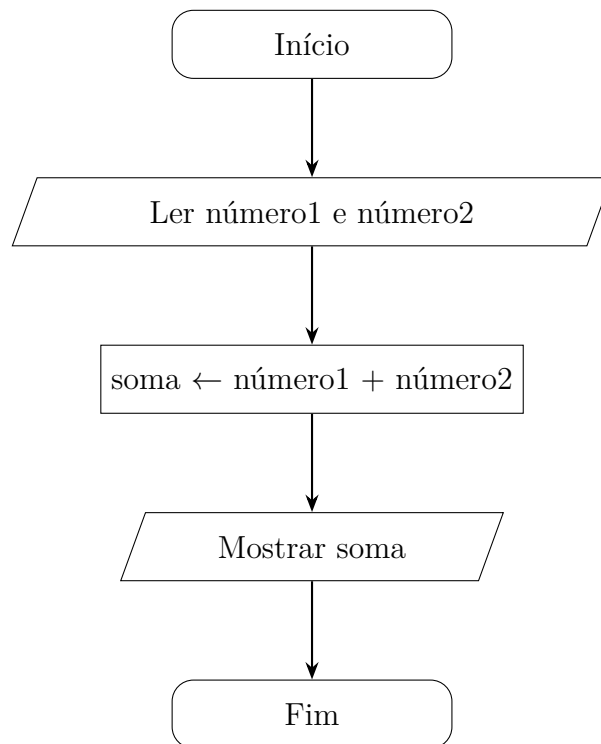


Figura 5.2: Fluxograma para a soma de dois números.

5.21 Exemplo 2 – Média de Duas Notas

Problema

Ler duas notas e calcular a média aritmética.

Análise

Entrada: nota1 e nota2;

Processamento: calcular

$$media = \frac{nota1 + nota2}{2};$$

Saída: média calculada.

Algoritmo

```
algoritmo "media"
```

```
var
```

```
    nota1, nota2, media: real
```

inicio

```
escreva("Digite a primeira nota: ")
leia(nota1)
```

```
escreva("Digite a segunda nota: ")
leia(nota2)
```

```
media <- (nota1 + nota2) / 2
```

```
escreval("Media = ", media:4:2)
```

fimalgoritmo

5.22 Exemplo 3 – Área de um Retângulo

Problema

Ler a base e a altura de um retângulo e calcular sua área.

A fórmula utilizada é:

$$A = b \times h$$

Algoritmo

algoritmo "area_retangulo"

var

```
base, altura, area: real
```

inicio

```
escreva("Digite a base: ")
leia(base)
```

```
escreva("Digite a altura: ")
leia(altura)
```

```
area <- base * altura
```

```
escreval("Area = ", area)
```

fimalgoritmo

5.23 Exemplo 4 – Conversão de Temperatura

Considere o problema de converter uma temperatura em graus Celsius para Fahrenheit. Utilizaremos:

$$F = \frac{9 \times C}{5} + 32$$

Entrada

Temperatura em Celsius.

Processamento

Aplicação da fórmula de conversão.

Saída

Temperatura em Fahrenheit.

Algoritmo

```
algoritmo "celsius_fahrenheit"

var
    celsius, fahrenheit: real

inicio

    escreva("Temperatura em Celsius: ")
    leia(celsius)

    fahrenheit <- (9 * celsius) / 5 + 32

    escreval("Temperatura em Fahrenheit: ",
            fahrenheit:6:2)

finalgoritmo
```

5.24 Exemplo 5 – Preço Final

Considere: Ler o preço de um produto e calcular um novo preço acrescentando R\$ 10,00.

Análise

Entrada: preço original;

Processamento: adicionar 10;

Saída: preço final.

Algoritmo

```
algoritmo "preco_final"

var
    preco, precoFinal: real

inicio

    escreva("Digite o preco do produto: ")
    leia(preco)

    precoFinal <- preco + 10

    escreval("Preco final = R$ ", precoFinal:6:2)

finalgoritmo
```

5.25 Planejando Antes de Programar

Antes de escrever o código, podemos construir uma pequena tabela de análise. Considere o problema da média de duas notas.

Tabela 5.2: Análise do problema da média.

Elemento	Identificação
Entrada	nota1, nota2
Processamento	$media \leftarrow (nota1 + nota2) / 2$
Saída	media

Essa técnica é especialmente útil quando o estudante ainda está aprendendo a transformar um problema em algoritmo.

5.26 Erros Comuns

5.26.1 Utilizar uma variável sem declaração

Inadequado:

```
algoritmo "erro"
```

inicio

leia(idade)

fimalgoritmo

O correto é declarar:

var

idade: inteiro

5.26.2 Esquecer a leitura

Considere:

var

numero: real

inicio

dobro <- numero * 2

fimalgoritmo

Se o objetivo era receber um valor do usuário, faltou:

leia(numero)

5.26.3 Confundir texto com variável

Considere:

nome <- "Ana"

Se utilizarmos:

escreval("nome")

será exibido:

nome

Para apresentar o conteúdo da variável:

escreval(nome)

5.26.4 Não orientar a entrada

Menos adequado:

```
leia(valor)
```

Mais claro:

```
escreva("Digite um valor: ")
leia(valor)
```

5.26.5 Utilizar uma fórmula incorreta

Considere o cálculo da média:

```
media <- nota1 + nota2 / 2
```

Devido à precedência das operações, essa expressão não calcula corretamente a média das duas notas.

O correto é:

```
media <- (nota1 + nota2) / 2
```

5.27 Estrutura Geral de um Algoritmo Sequencial

Podemos utilizar o seguinte modelo como referência inicial:

```
algoritmo "nome"

var
    // variaveis

inicio

    // entrada
    escreva("...")
    leia(...)

    // processamento
    resultado <- ...

    // saida
    escreval("Resultado = ", resultado)

finalgoritmo
```

Essa estrutura não é uma regra obrigatória para todos os algoritmos, mas representa bem muitos problemas introdutórios.

5.28 Resumo do Capítulo

Neste capítulo estudamos o modelo:

Entrada → Processamento → Saída

A **entrada** corresponde aos dados necessários para resolver um problema. No VisuAlg, a entrada pode ser realizada utilizando:

```
leia(...)
```

O **processamento** corresponde às operações realizadas sobre os dados. A **saída** corresponde aos resultados apresentados ao usuário. Os principais comandos utilizados são:

```
escreva(...)  
escreval(...)
```

Também vimos que algoritmos sequenciais executam suas instruções na ordem em que elas aparecem e que o teste de mesa pode ser utilizado para acompanhar os valores das variáveis durante a execução.

5.29 Atividades

Questões conceituais

1. O que significa o modelo Entrada–Processamento–Saída?
2. O que é entrada de dados?
3. O que é processamento?
4. O que é saída de dados?
5. Qual comando do VisuAlg é utilizado para entrada de dados?
6. Qual é a finalidade do comando `escreva`?
7. Qual é a finalidade do comando `escreval`?
8. Qual é a diferença entre `escreva` e `escreval`?
9. Por que é recomendável apresentar uma mensagem antes de utilizar `leia`?
10. O que caracteriza um algoritmo sequencial?
11. Qual é a finalidade de um teste de mesa?

Identifique Entrada, Processamento e Saída

Para cada problema a seguir, identifique os dados de entrada, o processamento e a saída.

1. Ler dois números e calcular sua soma.
2. Ler três notas e calcular a média.
3. Ler base e altura e calcular a área de um retângulo.
4. Ler o preço de um produto e calcular o preço com acréscimo de 15 reais.
5. Ler uma temperatura em Celsius e convertê-la para Fahrenheit.
6. Ler a idade de uma pessoa e calcular sua idade no próximo ano.
7. Ler o salário de um funcionário e calcular um novo salário com acréscimo de R\$ 200,00.

Construa os Algoritmos

Escreva algoritmos em Portugol para os problemas seguintes.

1. Ler um número e apresentar seu dobro.
2. Ler um número e apresentar seu triplo.
3. Ler dois números e apresentar a soma.
4. Ler dois números e apresentar a soma e a diferença.
5. Ler três notas e calcular a média aritmética.
6. Ler base e altura e calcular a área de um retângulo.
7. Ler o lado de um quadrado e calcular sua área.
8. Ler um valor em metros e convertê-lo para centímetros.
9. Ler uma quantidade de horas e calcular a quantidade correspondente de minutos.
10. Ler o preço de um produto e calcular o valor após um acréscimo de R\$ 20,00.

Complete o Algoritmo

Complete as partes ausentes.

```
algoritmo "dobro"

var
    numero, dobro: real

inicio
```

```
    escreva("Digite um numero: ")
    -----

    dobro <- -----

    escreval("Dobro = ", -----)

finalgoritmo
```

Encontre os Erros

Analise o algoritmo:

```
algoritmo "media"

var
    nota1, nota2: real
inicio

    escreva("Nota 1: ")
    leia(nota1)

    escreva("Nota 2: ")
    leia(nota2)

    media <- nota1 + nota2 / 2

    escreval("Media = ", "media")

finalgoritmo
```

Identifique os problemas existentes e reescreva corretamente o algoritmo.

5.30 Desafio

Desafio – Consumo de Combustível

Um motorista deseja calcular o consumo médio de combustível de um veículo. O algoritmo deverá receber:

- a distância percorrida em quilômetros;
- a quantidade de combustível consumida em litros.

O consumo médio pode ser calculado por:

$$consumo = \frac{distancia}{litros}$$

Construa:

1. a identificação da entrada, processamento e saída;
2. o fluxograma correspondente;
3. o algoritmo em Portugol;
4. a saída formatada com duas casas decimais.

Capítulo 6

Estruturas Condicionais

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender o conceito de tomada de decisão em algoritmos;
- identificar situações em que o fluxo de execução depende de uma condição;
- construir expressões lógicas para controlar decisões;
- utilizar corretamente a estrutura condicional simples;
- utilizar corretamente a estrutura condicional composta;
- construir estruturas condicionais aninhadas ou encadeadas;
- combinar operadores relacionais e lógicos em condições;
- utilizar a estrutura de seleção múltipla;
- representar estruturas condicionais por meio de fluxogramas;
- realizar testes de mesa envolvendo decisões;
- escolher a estrutura condicional mais adequada para cada problema.

Nos capítulos anteriores, os algoritmos estudados possuíam um fluxo predominantemente sequencial. Isso significa que as instruções eram executadas uma após a outra, normalmente de cima para baixo. Considere:

```
leia(nota1)
leia(nota2)

media <- (nota1 + nota2) / 2

escreval(media)
```

Nesse exemplo, todas as instruções são executadas. Entretanto, muitos problemas exigem que determinadas ações sejam realizadas apenas quando uma condição for satisfeita. Por exemplo:

- se a média for maior ou igual a 7, informar que o estudante foi aprovado;
- se a idade for maior ou igual a 18, informar que a pessoa é maior de idade;
- se um número for positivo, apresentar uma determinada mensagem;
- se uma senha estiver correta, permitir o acesso;
- caso contrário, executar uma ação diferente.

Para representar situações como essas utilizamos as **estruturas condicionais**.

Ideia central

Uma estrutura condicional permite que o algoritmo escolha qual caminho deverá ser seguido de acordo com o resultado de uma condição.

6.1 Tomada de Decisão

Uma **decisão** ocorre quando o fluxo de execução de um algoritmo depende do resultado de uma determinada condição. Considere a pergunta:

`idade >= 18`

Essa expressão possui apenas dois resultados possíveis:

VERDADEIRO ou FALSO

Se a expressão for verdadeira, o algoritmo poderá executar determinada ação. Caso seja falsa, poderá continuar sua execução ou seguir outro caminho. A decisão, portanto, depende de uma **expressão lógica**.

6.2 Condições

Uma condição utilizada em uma estrutura condicional deve produzir um resultado lógico. Por exemplo:

`idade >= 18`

ou:

`media >= 7`

ou ainda:

`(media >= 7) e (frequencia >= 75)`

Nos exemplos acima são utilizados operadores relacionais e lógicos estudados anteriormente. Os operadores relacionais mais utilizados são:

= <> > < >= <=

Enquanto alguns operadores lógicos são:

e ou nao

Importante

A estrutura condicional não decide por conta própria. Ela depende de uma expressão cujo resultado seja VERDADEIRO ou FALSO.

6.3 Estrutura Condicional Simples

A **estrutura condicional simples** é utilizada quando uma ou mais instruções devem ser executadas somente se determinada condição for verdadeira. No VisuAlg, sua forma geral é:

```
se <condicao> entao
```

```
    <comandos>
```

```
fimse
```

O funcionamento pode ser descrito da seguinte maneira:

1. a condição é avaliada;
2. se o resultado for VERDADEIRO, os comandos entre **entao** e **fimse** são executados;
3. se o resultado for FALSO, esses comandos são ignorados;
4. a execução continua após o **fimse**.

6.4 Exemplo de Condicional Simples

Considere o problema: **Ler a idade de uma pessoa e apresentar uma mensagem caso ela possua 18 anos ou mais.** O algoritmo pode ser escrito como:

```
algoritmo "maior_idade"
```

```
var
```

```
    idade: inteiro
```

```
inicio
```

```
    escreva("Digite sua idade: ")
```

```
    leia(idade)
```

```
se idade >= 18 entao  
    escreval("Voce possui 18 anos ou mais.")  
fimse
```

```
fimalgoritmo
```

Se o usuário informar:

20

a condição:

`idade >= 18`

será verdadeira e a mensagem será apresentada. Se o usuário informar:

16

a condição será falsa e nenhuma mensagem existente dentro da estrutura **se** será executada.

6.5 Fluxograma da Condicional Simples

A Figura 6.1 representa graficamente esse comportamento.

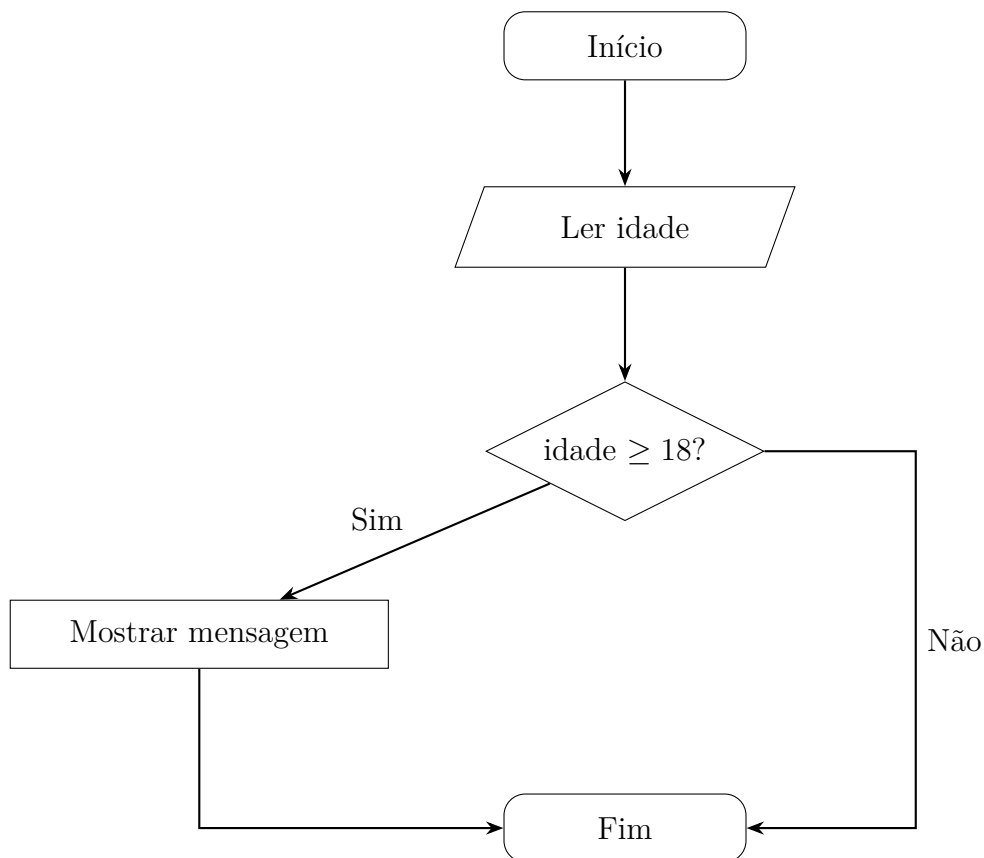


Figura 6.1: Fluxograma de uma estrutura condicional simples.

6.6 Exemplo 2 – Número Positivo

Considere:

Ler um número e informar quando ele for positivo.

```
algoritmo "numero_positivo"

var
    numero: real

inicio

    escreva("Digite um numero: ")
    leia(numero)

    se numero > 0 entao
        escreval("O numero e positivo.")
    fimse

finalgoritmo
```

Observe que, caso o número seja zero ou negativo, nenhuma mensagem será apresentada pela estrutura condicional.

6.7 Estrutura Condicional Composta

Em alguns problemas, precisamos executar uma ação quando a condição for verdadeira e outra ação quando ela for falsa. Nesse caso utilizamos a **estrutura condicional composta**. Sua forma geral no VisuAlg é:

```
se <condicao> entao

    <comandos_se_verdadeiro>

senao

    <comandos_se_falso>

fimse
```

Nessa estrutura, apenas um dos dois caminhos será executado.

6.8 Funcionamento da Condicional Composta

Considere:

```
se media >= 7 entao
    escreval("Aprovado")

senao
    escreval("Reprovado")

fimse
```

Se:

```
media >= 7
    for verdadeiro, será executado:
        escreval("Aprovado")

    Caso contrário, será executado:
        escreval("Reprovado")
```

Observe

Em uma estrutura `se...senao`, apenas um dos blocos é executado em cada avaliação da condição.

6.9 Exemplo 3 – Aprovação

Considere, apenas para fins didáticos, que um estudante seja considerado aprovado quando sua média for maior ou igual a 7.

```
algoritmo "situacao_aluno"

var
    nota1, nota2, media: real

inicio

    escreva("Digite a primeira nota: ")
    leia(nota1)

    escreva("Digite a segunda nota: ")
    leia(nota2)
```

```
media <- (nota1 + nota2) / 2

escreval("Media = ", media:4:2)

se media >= 7 entao
  escreval("Situacao: APROVADO")
senao
  escreval("Situacao: REPROVADO")
fimse

finalgoritmo
```

6.10 Fluxograma da Condicional Composta

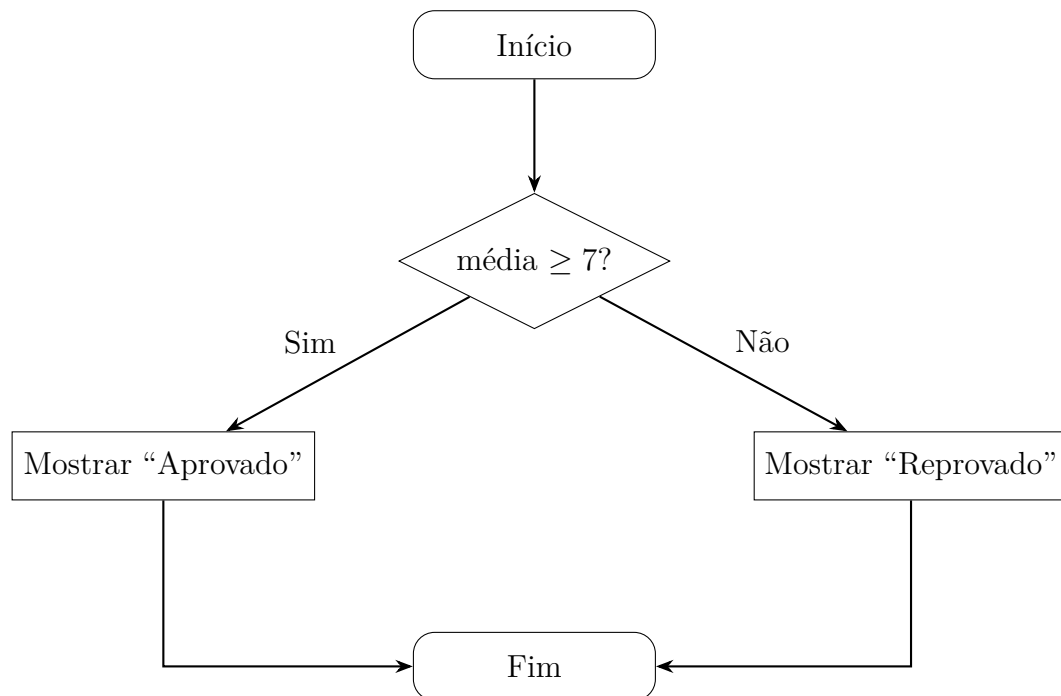


Figura 6.2: Fluxograma de uma estrutura condicional composta.

6.11 Exemplo 4 – Número Par ou Ímpar

O operador de resto da divisão pode ser utilizado para verificar se um número inteiro é par. Se:

```
numero % 2 = 0
```

então o número é divisível por 2. Podemos escrever:

```
algoritmo "par_impar"

var
    numero: inteiro

inicio

    escreva("Digite um numero inteiro: ")
    leia(numero)

    se numero % 2 = 0 entao
        escreval("O numero e par.")
    senao
        escreval("O numero e impar.")
    fimse

finalgoritmo
```

6.12 Utilizando Operadores Lógicos nas Condições

Uma condição pode combinar mais de uma expressão lógica. Considere:

$(media \geq 7)$ e $(frequencia \geq 75)$

Nesse caso, as duas condições precisam ser verdadeiras. Podemos escrever:

```
se (media >= 7) e (frequencia >= 75) entao

    escreval("Criterios atendidos.")

senao

    escreval("Criterios nao atendidos.")

fimse
```

O uso de expressões lógicas compostas em estruturas condicionais permite representar regras mais próximas de problemas reais.

6.13 Exemplo 5 – Valor Dentro de um Intervalo

Considere o problema: **Verificar se uma nota está entre 0 e 10.** A condição pode ser escrita como:

$(nota \geq 0)$ e $(nota \leq 10)$

Assim:

```
algoritmo "validar_nota"

var
    nota: real

inicio

    escreva("Digite uma nota: ")
    leia(nota)

    se (nota >= 0) e (nota <= 10) entao
        escreval("Nota valida.")
    senao
        escreval("Nota invalida.")
    fimse

fimalgoritmo
```

6.14 Condicionais Aninhadas

Uma estrutura condicional pode ser colocada dentro de outra estrutura condicional. Essa organização é denominada **condicional aninhada**.

Também encontramos na literatura a expressão **estrutura condicional encadeada** quando diversas decisões são organizadas hierarquicamente. O VisuAlg permite esse tipo de aninhamento. A forma geral pode ser representada por:

```
se condicao1 entao

    se condicao2 entao
        comandos
    senao
        comandos
    fimse

senao

    comandos

fimse
```

Nesse caso, a segunda condição somente será avaliada quando o fluxo alcançar a estrutura interna.

6.15 Exemplo 6 – Situação do Estudante

Considere, apenas como exemplo didático, os seguintes critérios:

- frequência inferior a 75: reprovado por frequência;
- frequência adequada e média maior ou igual a 7: aprovado;
- frequência adequada e média maior ou igual a 3, porém menor que 7: recuperação;
- frequência adequada e média menor que 3: reprovado por média.

Uma possível solução é:

```
algoritmo "situacao_estudante"

var
    media, frequencia: real

inicio

    escreva("Digite a media: ")
    leia(media)

    escreva("Digite a frequencia: ")
    leia(frequencia)

    se frequencia >= 75 entao

        se media >= 7 entao

            escreval("Situacao: APROVADO")

        senao

            se media >= 3 entao

                escreval("Situacao: RECUPERACAO")

            senao

                escreval("Situacao: REPROVADO POR MEDIA")

            fimse

        fimse

    fimse

    escreval("Situacao: REPROVADO POR FREQUENCIA")

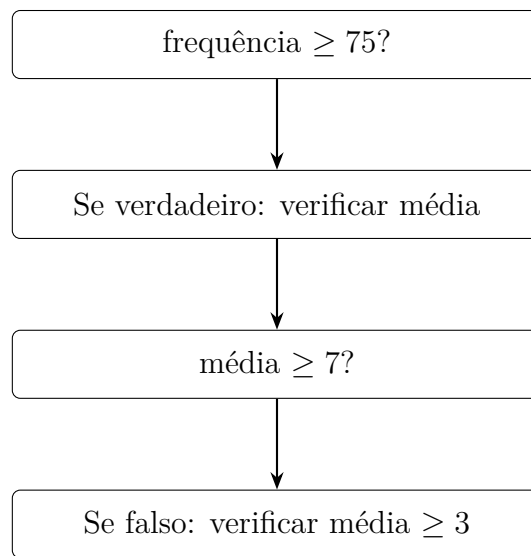
    fimse

finalgoritmo
```

Observe que uma segunda decisão é realizada somente quando a primeira condição permite continuar a análise.

6.16 Visualizando o Aninhamento

Podemos representar a estrutura anterior de maneira simplificada:



Quanto maior o número de decisões aninhadas, maior deve ser o cuidado com a organização e a indentação do algoritmo.

6.17 Indentação

A indentação consiste em deslocar visualmente comandos que pertencem a determinado bloco.

Compare:

```
se idade >= 18 entao
  escreval("Maior de idade")
senao
  escreval("Menor de idade")
fimse
```

com:

```
se idade >= 18 entao
  escreval("Maior de idade")
senao
  escreval("Menor de idade")
fimse
```

As duas formas podem representar a mesma lógica, mas a segunda é muito mais fácil de ler. Em estruturas aninhadas, a indentação torna-se ainda mais importante.

Boa prática

Utilize indentação para mostrar visualmente quais comandos pertencem a cada estrutura condicional.

6.18 Condicionais Independentes

Nem sempre duas decisões precisam estar aninhadas. Considere:

```
se idade >= 18 entao
    escreval("Maior de idade")
fimse

se nota >= 7 entao
    escreval("Nota satisfatoria")
fimse
```

As duas condições são avaliadas independentemente. Isso é diferente de:

```
se idade >= 18 entao

    se nota >= 7 entao
        escreval("Maior de idade e nota satisfatoria")
    fimse

fimse
```

No segundo caso, a condição relacionada à nota somente será verificada quando `idade >= 18` for verdadeira.

Alguns materiais denominam estruturas independentes colocadas em sequência como **seleções concatenadas**, distinguindo-as das seleções aninhadas.

6.19 Condicionais Encadeadas

É comum encontrarmos problemas nos quais uma variável precisa ser classificada em mais de duas situações. Considere: **Classificar uma temperatura como baixa, moderada ou alta**. Uma possível solução é:

```
se temperatura < 20 entao

    escreval("Temperatura baixa")

senao
```

```
se temperatura < 30 entao
    escreval("Temperatura moderada")
senao
    escreval("Temperatura alta")
fimse

fimse
```

Observe que, a cada caminho falso, uma nova condição é avaliada.

6.20 Ordem das Condições

Em estruturas encadeadas, a ordem das condições pode modificar completamente o resultado. Considere a classificação:

- média maior ou igual a 9: excelente;
- média maior ou igual a 7: boa;
- caso contrário: insuficiente.

Uma forma adequada é:

```
se media >= 9 entao
    escreval("Excelente")
senao
    se media >= 7 entao
        escreval("Boa")
    senao
        escreval("Insuficiente")
    fimse
fimse

fimse
```

Se testássemos primeiro:

```
media >= 7
```

uma média igual a 9 já satisfaria essa condição e poderia ser classificada antes que o teste mais específico fosse realizado.

Atenção

Em classificações por faixa, analise cuidadosamente a ordem em que as condições serão avaliadas.

6.21 Seleção Múltipla

Quando precisamos comparar uma mesma expressão com vários valores possíveis, uma sequência muito grande de estruturas condicionais pode dificultar a leitura. Para determinadas situações, o VisuAlg oferece a estrutura:

`escolha`

A forma geral é:

`escolha <expressao>`

`caso <valor1>`
`<comandos>`

`caso <valor2>`
`<comandos>`

`outrocaso`
`<comandos>`

`fimescolha`

A expressão é avaliada e o bloco correspondente ao valor encontrado é executado.

6.22 Exemplo 7 – Opções de um Menu

Considere:

`algoritmo "menu"`

`var`

`opcao: inteiro`

`inicio`

`escreval("1 - Cadastrar")`
`escreval("2 - Consultar")`
`escreval("3 - Sair")`

`escreva("Escolha uma opcao: ")`
`leia(opcao)`

`escolha opcao`

`caso 1`
`escreval("Opcao: Cadastrar")`

```
        caso 2
            escreval("Opcao: Consultar")

        caso 3
            escreval("Opcao: Sair")

        outrocaso
            escreval("Opcao invalida")

    fimescolha

finalgoritmo
```

6.23 Vários Valores em um Mesmo Caso

O VisuAlg permite associar mais de um valor ao mesmo bloco. Por exemplo:

```
escolha numero

    caso 1, 3, 5, 7, 9
        escreval("Numero impar entre 1 e 9")

    caso 2, 4, 6, 8
        escreval("Numero par entre 1 e 9")

    outrocaso
        escreval("Valor fora da classificacao")

fimescolha
```

Essa característica pode tornar determinadas classificações mais compactas.

6.24 se ou escolha?

As estruturas não são equivalentes em todas as situações. O comando **se** é bastante adequado quando precisamos avaliar:

- intervalos;
- desigualdades;
- expressões relacionais;
- condições compostas;
- combinações com operadores lógicos.

Por exemplo:

```
se idade >= 18 entao
```

Já `escolha` é especialmente útil quando uma expressão deve ser comparada com valores específicos. Por exemplo:

```
escolha opcao
```

```
  caso 1
```

```
  caso 2
```

```
  caso 3
```

```
fimescolha
```

6.25 Erros Comuns

6.25.1 Confundir atribuição com comparação

Inadequado como condição:

```
se idade <- 18 entao
```

A comparação correta é:

```
se idade = 18 entao
```

O operador:

```
<-
```

é utilizado para atribuição.

6.25.2 Esquecer o `fimse`

Inadequado:

```
se idade >= 18 entao  
  escreval("Maior de idade")
```

Adequado:

```
se idade >= 18 entao  
  escreval("Maior de idade")  
fimse
```

6.25.3 Escrever **senao** em uma condicional simples

Se nenhuma ação precisa ocorrer quando a condição for falsa, o **senao** não é necessário. Por exemplo:

```
se temperatura > 40 entao
    escreval("Temperatura elevada")
fimse
```

6.25.4 Condições incompletas

Considere a necessidade de verificar se uma nota está entre 0 e 10. Inadequado:

```
se nota >= 0 entao
```

Essa condição não verifica o limite superior. Uma forma adequada seria:

```
se (nota >= 0) e (nota <= 10) entao
```

6.25.5 Intervalos sobrepostos

Ao utilizar várias condições, é importante verificar se as faixas foram definidas corretamente. Por exemplo:

```
se idade >= 10 entao
    ...
senao
    se idade >= 15 entao
        ...
    fimse
fimse
```

Nesse caso, quando a idade for 15, a primeira condição já será verdadeira. A segunda nunca será alcançada para esse valor. A ordem das condições precisa ser planejada.

6.26 Escolhendo a Estrutura Adequada

Uma estratégia simples pode ajudar na escolha.

Use uma **condicional simples** quando:

uma ação deve ocorrer apenas quando a condição for verdadeira.

Use uma **condicional composta** quando:

existem duas alternativas: uma para verdadeiro e outra para falso.

Use **condicionais aninhadas ou encadeadas** quando:

uma decisão depender de outra ou houver várias classificações sucessivas.

Use **escolha...caso** quando:

uma mesma expressão precisa ser comparada com vários valores específicos.

6.27 Resumo do Capítulo

Neste capítulo estudamos as estruturas utilizadas para modificar o fluxo de execução de um algoritmo de acordo com determinadas condições. A estrutura condicional simples possui a forma:

```
se condicao entao
    comandos
fimse
```

e executa seus comandos somente quando a condição é verdadeira. A estrutura condicional composta utiliza:

```
se condicao entao
    comandos
senao
    comandos
fimse
```

permitindo escolher entre dois caminhos. Também vimos que estruturas condicionais podem ser colocadas dentro de outras estruturas, formando condicionais aninhadas ou encadeadas. Condições podem utilizar operadores relacionais e lógicos.

Finalmente, estudamos a estrutura:

```
escolha ... caso ... outrocaso ... fimescolha
```

adequada para determinadas situações de seleção múltipla.

6.28 Atividades

Questões conceituais

1. O que é uma estrutura condicional?
2. Qual é a função de uma condição?
3. Que tipo de resultado uma condição deve produzir?
4. Qual é a diferença entre uma estrutura condicional simples e uma estrutura condicional composta?
5. Quando o bloco de uma estrutura condicional simples é executado?
6. Qual é a função do comando **senao**?
7. O que é uma estrutura condicional aninhada?
8. Qual é a importância da indentação em estruturas aninhadas?
9. O que é uma condição composta?
10. Em quais situações a estrutura **escolha** pode ser útil?
11. Qual é a função de **outrocaso**?

Determine o Resultado

Considere:

a ← 10

b ← 5

Determine quais mensagens serão apresentadas.

1. se a > b entao
 escreval("A")
fimse
2. se a = b entao
 escreval("X")
senao
 escreval("Y")
fimse
3. se (a > 5) e (b < 10) entao
 escreval("V")
senao
 escreval("F")
fimse

Construa as Condições

Escreva a expressão lógica necessária para representar cada situação.

1. idade maior ou igual a 18;
2. número menor que zero;
3. média maior ou igual a 7;
4. nota entre 0 e 10;
5. idade entre 15 e 17;
6. média maior ou igual a 7 e frequência maior ou igual a 75;
7. dia igual a sábado ou domingo;
8. número diferente de zero.

Condicional Simples

Construa algoritmos para:

1. ler um número e apresentar uma mensagem caso seja positivo;
2. ler uma idade e informar caso seja maior ou igual a 18;
3. ler uma nota e informar quando ela for igual a 10;
4. ler um número inteiro e informar quando ele for divisível por 5.

Condicional Composta

Construa algoritmos para:

1. ler um número e informar se ele é positivo ou negativo;
2. ler um número inteiro e informar se ele é par ou ímpar;
3. ler duas notas, calcular a média e informar “Aprovado” quando a média for maior ou igual a 7 e “Reprovado” caso contrário;
4. ler dois números e informar qual deles é maior;
5. ler uma idade e informar “Maior de idade” ou “Menor de idade”.

Condicionais Encadeadas

Construa algoritmos para:

1. ler um número e classificá-lo como positivo, negativo ou zero;
2. ler uma média e classificá-la em três faixas definidas pelo professor;
3. ler a idade de uma pessoa e classificá-la em três faixas etárias;
4. ler três valores e determinar o maior deles.

Seleção Múltipla

Utilize `escolha...caso` para:

1. ler um número de 1 a 3 e apresentar:
 - 1 – Incluir;
 - 2 – Consultar;
 - 3 – Sair.
2. ler um número de 1 a 7 e apresentar o nome correspondente ao dia da semana;
3. ler uma opção de menu e apresentar a operação selecionada.

Encontre os Erros

Analise:

```
algoritmo "teste"
```

```
var
```

```
    idade: inteiro
```

```
inicio
```

```
escreva("Digite a idade: ")
leia(idade)

se idade <- 18 entao
    escreval("Idade igual a 18")
senao
    escreval("Idade diferente de 18")

finalgoritmo
```

Identifique os erros e apresente a versão correta.

6.29 Desafio

Desafio – Classificação de uma Compra

Uma loja deseja classificar o valor total de uma compra utilizando os seguintes critérios didáticos:

- valor inferior a R\$ 100,00: compra pequena;
- valor entre R\$ 100,00 e R\$ 499,99: compra média;
- valor igual ou superior a R\$ 500,00: compra grande.

Construa:

1. a identificação da entrada e da saída;
2. as condições necessárias;
3. o fluxograma correspondente;
4. o algoritmo utilizando estruturas condicionais;
5. um teste de mesa para os valores:
 - R\$ 80,00;
 - R\$ 250,00;
 - R\$ 700,00.

Capítulo 7

Estruturas de Repetição

Objetivos de aprendizagem

Ao final deste capítulo, espera-se que o estudante seja capaz de:

- compreender a finalidade das estruturas de repetição;
- reconhecer situações que exigem a repetição de comandos;
- compreender o conceito de laço e iteração;
- utilizar a estrutura `enquanto...faca`;
- utilizar a estrutura `repita...ate`;
- utilizar a estrutura `para...faca`;
- diferenciar teste no início e teste no fim;
- utilizar variáveis de controle;
- construir contadores;
- construir acumuladores;
- reconhecer e evitar laços infinitos;
- escolher a estrutura de repetição mais adequada para cada problema;
- construir repetições aninhadas;
- realizar testes de mesa envolvendo estruturas de repetição.

Em muitos problemas computacionais, uma mesma instrução ou conjunto de instruções precisa ser executado várias vezes. Considere, por exemplo, a necessidade de apresentar os números de 1 até 10. Sem utilizar uma estrutura de repetição, poderíamos escrever:

```
escreval(1)
escreval(2)
escreval(3)
escreval(4)
escreval(5)
```

```
escreval(6)
escreval(7)
escreval(8)
escreval(9)
escreval(10)
```

Embora funcione, essa solução apresenta muita repetição de código. Se desejássemos mostrar os números de 1 até 1000, essa estratégia se tornaria completamente inadequada. Para resolver problemas desse tipo utilizamos as **estruturas de repetição**.

Ideia central

Uma estrutura de repetição permite executar um bloco de comandos várias vezes sem precisar escrever novamente esses comandos.

7.1 Laços de Repetição

Uma estrutura de repetição também pode ser chamada de:

- **laço**;
- **laço de repetição**;
- **loop**.

Cada execução do bloco de comandos existente dentro do laço é chamada de **iteração**. Considere:

```
escreval("Algoritmos")
```

Se essa instrução for executada cinco vezes dentro de uma estrutura de repetição, dizemos que o laço realizou cinco iterações.

7.2 Quando Utilizar uma Repetição?

Estruturas de repetição podem ser utilizadas em situações como:

- apresentar os números de 1 até 100;
- ler as notas de vários estudantes;
- calcular a soma de diversos valores;
- contar quantos estudantes foram aprovados;
- apresentar uma tabuada;
- repetir uma operação até que o usuário escolha sair;
- solicitar novamente um dado enquanto ele for inválido.

A quantidade de repetições pode ser:

- conhecida antecipadamente;
- determinada durante a execução;
- dependente de uma condição.

7.3 Estruturas de Repetição no VisuAlg

O VisuAlg apresenta três estruturas fundamentais de repetição:

1. enquanto...faca;
2. repita...ate;
3. para...ate...faca.

Cada estrutura possui características próprias.

Tabela 7.1: Principais estruturas de repetição utilizadas nesta apostila.

Estrutura	Tipo	Característica principal
enquanto...faca	Teste no início	A condição é avaliada antes da execução do bloco.
repita...ate	Teste no fim	A condição é avaliada depois da execução do bloco.
para...faca	Variável de controle	Adequada quando a quantidade de repetições é conhecida ou determinada por limites.

7.4 Repetição com Teste no Início

A estrutura:

```
enquanto...faca
```

realiza o teste da condição **antes** da execução dos comandos do laço. Sua forma geral é:

```
enquanto <condicao> faca
```

```
    <comandos>
```

```
fimenquanto
```

O funcionamento ocorre da seguinte maneira:

1. a condição é avaliada;
2. se a condição for VERDADEIRO, o bloco é executado;
3. ao chegar ao **fimenquanto**, o fluxo retorna para o teste;
4. a condição é avaliada novamente;
5. quando a condição for FALSO, o laço termina.

7.5 Exemplo com enquanto

Considere o problema: **Apresentar os números de 1 até 10**. Uma possível solução é:
algoritmo "numeros_1_a_10"

```
var
    numero: inteiro

inicio

    numero <- 1

    enquanto numero <= 10 faca

        escreval(numero)

        numero <- numero + 1

    fimenquanto

finalgoritmo
```

Nesse algoritmo, a variável `numero` controla a repetição.

7.6 Entendendo o Funcionamento

Inicialmente:

```
numero <- 1
```

A condição:

```
numero <= 10
```

é avaliada.

Como:

$$1 \leq 10$$

o bloco é executado.

Depois:

```
numero <- numero + 1
```

faz com que o valor passe de 1 para 2.

A condição é testada novamente:

$$2 \leq 10$$

O processo continua até que:

$$numero = 11$$

Nesse momento:

$$11 \leq 10$$

é falso e a repetição termina.

7.7 Elementos de um Laço Controlado por Condição

No exemplo anterior podemos identificar três elementos importantes:

1. **inicialização;**
2. **condição;**
3. **atualização.**

No algoritmo:

```
numero <- 1
```

```
enquanto numero <= 10 faca
```

```
  escreval(numero)
```

```
  numero <- numero + 1
```

```
fimenquanto
```

temos:

Tabela 7.2: Elementos de controle da repetição.

Elemento	Comando
Inicialização	<code>numero <- 1</code>
Condição	<code>numero <= 10</code>
Atualização	<code>numero <- numero + 1</code>

7.8 Fluxograma do enquanto

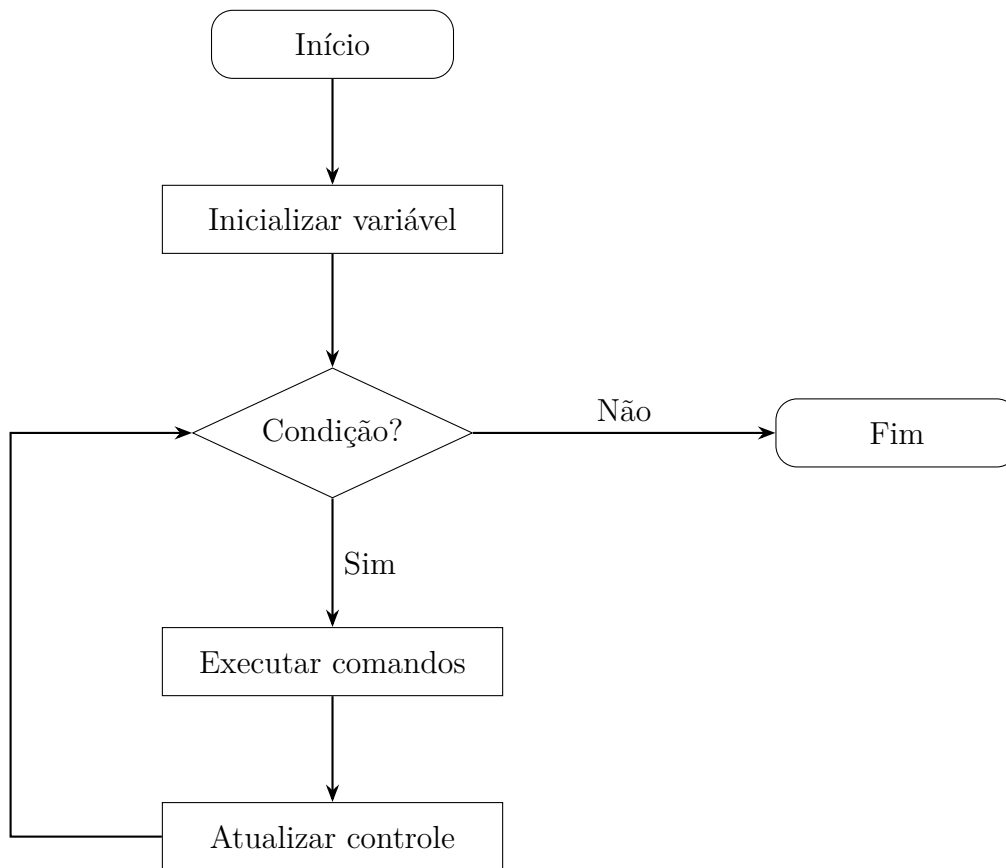


Figura 7.1: Fluxo básico de uma repetição com teste no início.

7.9 O Bloco Pode Não Ser Executado

Como o teste é realizado no início, existe a possibilidade de o bloco não ser executado nenhuma vez. Considere:

```
numero <- 20
```

```
enquanto numero <= 10 faca
```

```
    escreval(numero)
```

```
    numero <- numero + 1
```

```
fimenquanto
```

A primeira condição será:

$$20 \leq 10$$

que é falsa. Portanto, o bloco não será executado.

Característica do enquanto

Como a condição é testada antes do bloco, uma estrutura `enquanto...faca` pode executar seus comandos **zero ou mais vezes**.

7.10 Exemplo – Contagem Regressiva

```
algoritmo "contagem_regressiva"
```

```
var
```

```
    numero: inteiro
```

```
inicio
```

```
    numero <- 10
```

```
    enquanto numero >= 1 faca
```

```
        escreval(numero)
```

```
        numero <- numero - 1
```

```
    fimenquanto
```

```
    escreval("Fim!")
```

```
fimalgoritmo
```

Observe que nesse caso a variável é **decrementada**.

7.11 Laço Infinito

Um dos erros mais comuns ao utilizar estruturas de repetição é o **laço infinito**. Isso ocorre quando a condição nunca se torna falsa. Considere:

```
numero <- 1
```

```
enquanto numero <= 10 faca
```

```
    escreval(numero)
```

```
fimenquanto
```

A variável `numero` começa com 1, mas nunca é alterada. Assim:

```
numero <= 10
```

permanecerá verdadeiro indefinidamente. Uma forma correta seria:

```
numero <- 1

enquanto numero <= 10 faca

    escreval(numero)

    numero <- numero + 1

fimenquanto
```

Atenção

Quando utilizar uma repetição controlada por condição, verifique se alguma instrução existente no laço modifica os elementos necessários para que a condição possa, em algum momento, terminar a repetição.

7.12 Repetição com Teste no Fim

A estrutura:

```
repita...ate
```

realiza o teste da condição **depois** da execução do bloco. Sua forma geral é:

```
repita

    <comandos>

ate <condicao>
```

O funcionamento é:

1. os comandos são executados;
2. a condição é avaliada;
3. se a condição for **FALSO**, o bloco é executado novamente;
4. quando a condição for **VERDADEIRO**, a repetição termina.

7.13 A Condição do repita

É importante observar uma diferença conceitual. No:

```
enquanto condicao faca
```

a repetição continua enquanto a condição for verdadeira. No:

```
repita
...
ate condicao
```

a repetição termina quando a condição se torna verdadeira.

Atenção à condição

No **enquanto**, pensamos normalmente em uma **condição de continuação**.
No **repita...ate**, pensamos normalmente em uma **condição de término**.

7.14 Exemplo com repita

O exemplo dos números de 1 até 10 pode ser reescrito como:

```
algoritmo "numeros_1_a_10_repita"
```

```
var
    numero: inteiro
```

```
inicio
```

```
    numero <- 1
```

```
    repita
```

```
        escreval(numero)
```

```
        numero <- numero + 1
```

```
    ate numero > 10
```

```
fimalgoritmo
```

Observe que a condição de término é:

```
numero > 10
```

Quando essa expressão se torna verdadeira, a repetição termina.

7.15 O Bloco é Executado pelo Menos uma Vez

Considere:

```
numero <- 20
```

```
repita
```

```
    escreval(numero)
```

```
    numero <- numero + 1
```

```
ate numero > 10
```

Embora o valor inicial já seja maior do que 10, o bloco será executado antes do teste. Portanto, o valor 20 será apresentado.

Característica do `repita`

Como a condição é testada depois do bloco, a estrutura `repita...ate` executa seus comandos **uma ou mais vezes**.

7.16 Fluxograma do `repita`

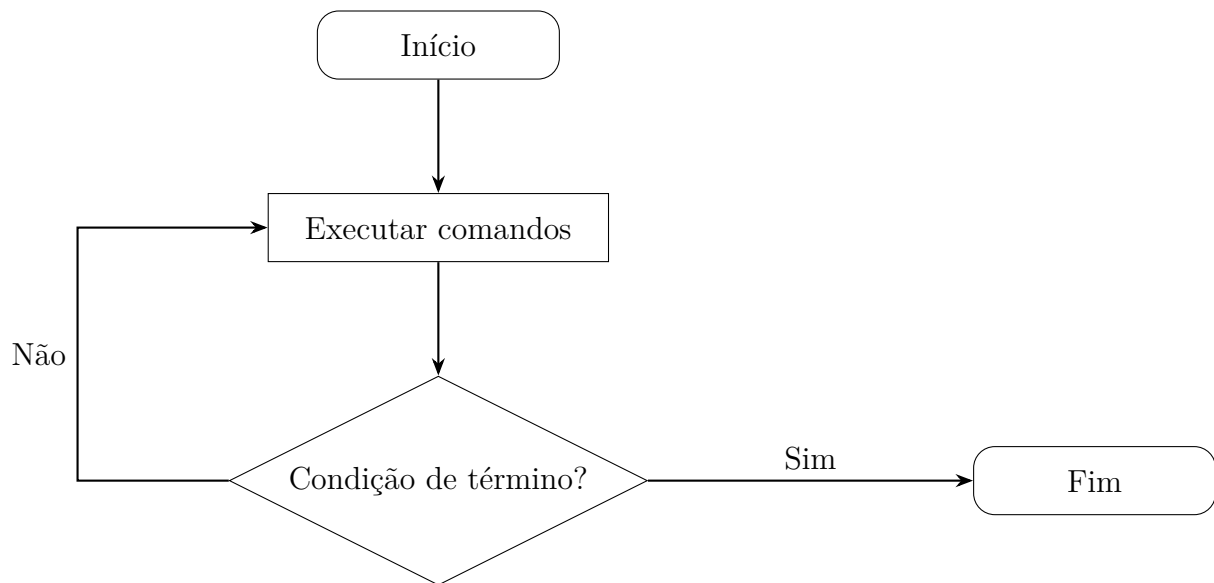


Figura 7.2: Fluxo básico de uma repetição com teste no fim.

7.17 Exemplo – Repetir até o Usuário Escolher Sair

A estrutura `repita...ate` é especialmente útil quando uma operação precisa acontecer pelo menos uma vez. Considere:

```
algoritmo "novo_calculo"
```

```
var
```

```
a, b, soma: real
resposta: caractere

inicio

    repita

        escreva("Digite o primeiro valor: ")
        leia(a)

        escreva("Digite o segundo valor: ")
        leia(b)

        soma <- a + b

        escreval("Soma = ", soma)

        escreva("Deseja realizar novo calculo? (S/N): ")
        leia(resposta)

    ate resposta = "N"

    escreval("Programa encerrado.")

finalgoritmo
```

Nesse problema, não sabemos antecipadamente quantas vezes o usuário desejará realizar o cálculo.

7.18 Comparando enquanto e repita

Tabela 7.3: Comparação entre **enquanto** e **repita**.

Característica	enquanto	repita
Local do teste	Antes do bloco	Depois do bloco
Número mínimo de execuções	Zero	Uma
Continua quando	A condição é verdadeira	A condição do ate é falsa
Termina quando	A condição fica falsa	A condição do ate fica verdadeira
Uso comum	Quando é necessário testar antes de executar	Quando o bloco precisa ocorrer ao menos uma vez

7.19 Repetição com Variável de Controle

Quando sabemos antecipadamente quantas vezes um conjunto de comandos deverá ser executado, podemos utilizar a estrutura:

```
para...faca
```

Sua forma geral é:

```
para <variavel> de <valor_inicial> ate <valor_final> faca
```

```
    <comandos>
```

```
fimpara
```

Também é possível especificar um passo:

```
para <variavel> de <inicio> ate <fim> passo <incremento> faca
```

```
    <comandos>
```

```
fimpara
```

O VisuAlg utiliza uma variável inteira para controlar as iterações dessa estrutura.

7.20 Exemplo com para

Para apresentar os números de 1 até 10:

```
algoritmo "numeros_para"
```

```
var
```

```
    i: inteiro
```

```
inicio
```

```
    para i de 1 ate 10 faca
```

```
        escreval(i)
```

```
    fimpara
```

```
fimalgoritmo
```

Nesse caso:

- i é a variável de controle;
- 1 é o valor inicial;
- 10 é o valor final;
- o incremento padrão é 1.

7.21 Variável de Controle

A variável utilizada no comando `para` assume automaticamente uma sequência de valores. No exemplo:

```
para i de 1 ate 5 faca
    escreval(i)
fimpara
```

a variável `i` assume sucessivamente:

1, 2, 3, 4, 5

Assim, o bloco é executado cinco vezes.

7.22 Utilizando passo

Quando o incremento é diferente de 1, podemos utilizar:

`passo`

Por exemplo:

```
para i de 0 ate 10 passo 2 faca
    escreval(i)
fimpara
```

A saída será:

0
2
4
6
8
10

7.23 Contagem Decrescente com `para`

O passo também pode ser negativo. Por exemplo:

```
para i de 10 ate 1 passo -1 faca
    escreval(i)
fimpara
```

A saída será:

```
10
9
8
7
6
5
4
3
2
1
```

A documentação do VisuAlg destaca que uma contagem decrescente exige um passo negativo; sem isso, um laço de 10 até 1 não executaria suas instruções.

7.24 Exemplo – Tabuada

Considere o problema:

Ler um número e apresentar sua tabuada de 1 até 10.

```
algoritmo "tabuada"

var
    numero, i, resultado: inteiro

inicio

    escreva("Digite um numero: ")
    leia(numero)

    para i de 1 ate 10 faca

        resultado <- numero * i

        escreval(numero, " x ", i, " = ", resultado)

    fimpara

finalgoritmo
```

7.25 Quando Utilizar o para?

O `para...faca` é particularmente adequado quando conhecemos os limites da repetição. Exemplos:

- ler 10 notas;
- apresentar os números de 1 até 100;
- repetir uma operação 20 vezes;
- gerar uma tabuada;
- percorrer posições numeradas de uma estrutura de dados.

O uso dessa estrutura é melhor aplicada quando a quantidade de repetições é conhecida durante a elaboração do algoritmo ou pode ser fornecida durante a execução.

7.26 Um Mesmo Problema com Diferentes Estruturas

O problema de apresentar os números de 1 até 5 pode ser resolvido de diferentes formas.

Com enquanto

```
i <- 1  
  
enquanto i <= 5 faca  
    escreval(i)  
    i <- i + 1  
  
fimenquanto
```

Com repita

```
i <- 1  
  
repita  
    escreval(i)  
    i <- i + 1  
  
ate i > 5
```

Com para

```
para i de 1 ate 5 faca  
    escreval(i)  
  
fimpara
```

As três soluções podem produzir o mesmo resultado, mas possuem formas diferentes de controlar a repetição.

7.27 Escolhendo a Estrutura de Repetição

Não existe uma única estrutura adequada para todos os problemas. Uma orientação inicial é:

Qual estrutura utilizar?

Utilize **para...faca** quando a quantidade de repetições for conhecida ou puder ser expressa claramente por um intervalo.

Utilize **enquanto...faca** quando a execução depender de uma condição que precisa ser verificada **antes** do bloco.

Utilize **repita...ate** quando o bloco precisar ser executado **pelo menos uma vez** antes da verificação da condição de término.

7.28 Contadores

Durante uma repetição, muitas vezes precisamos contar quantas vezes um evento ocorreu. Para isso utilizamos uma variável chamada **contador**. Um contador normalmente é inicializado com zero:

```
contador <- 0
```

e aumentado quando o evento desejado ocorre:

```
contador <- contador + 1
```

O contador é uma forma típica de contar ocorrências dentro de estruturas de repetição.

7.29 Exemplo – Quantidade de Aprovados

Considere, apenas para fins didáticos: **Ler a nota de 10 estudantes e contar quantos possuem nota maior ou igual a 7.**

```
algoritmo "contar_aprovados"
```

```
var
```

```
    i, aprovados: inteiro
```

```
    nota: real
```

```
inicio
```

```
    aprovados <- 0
```

```
para i de 1 ate 10 faca

    escreva("Digite a nota do estudante ", i, ": ")
    leia(nota)

    se nota >= 7 entao
        aprovados <- aprovados + 1
    fimse

fimpara

escreval("Quantidade de aprovados: ", aprovados)

fimalgoritmo
```

Observe que:

```
aprovados <- 0
```

ocorre antes da repetição.
Dentro do laço:

```
aprovados <- aprovados + 1
```

é executado somente quando a condição é satisfeita.

7.30 Por que Inicializar o Contador?

Antes de utilizar um contador precisamos definir seu valor inicial.
Normalmente:

```
contador <- 0
```

Isso significa que, antes de analisar qualquer elemento, foram encontradas zero ocorrências. Depois, cada ocorrência acrescenta uma unidade:

```
contador <- contador + 1
```

Assim, podemos ter a evolução:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$$

7.31 Acumuladores

Enquanto um contador normalmente registra a **quantidade de ocorrências**, um acumulador armazena progressivamente valores. Um acumulador utilizado para somatório normalmente é inicializado com zero:

```
soma <- 0
```

e atualizado por:

```
soma <- soma + valor
```

Os acumuladores são variáveis empregadas para armazenar progressivamente resultados de somatórios ou produtórios.

7.32 Contador e Acumulador

Considere os comandos:

```
contador <- contador + 1
```

e:

```
soma <- soma + nota
```

No primeiro:

- acrescentamos sempre uma unidade;
- estamos contando ocorrências.

No segundo:

- acrescentamos o valor de `nota`;
- estamos acumulando valores.

Tabela 7.4: Diferença entre contador e acumulador.

Tipo	Exemplo	Finalidade
Contador	<code>contador <- contador + 1</code>	Contar ocorrências.
Acumulador	<code>soma <- soma + valor</code>	Somar valores sucessivamente.

7.33 Exemplo – Soma de Vários Valores

Considere:

Ler cinco números e calcular sua soma.

```
algoritmo "somar_cinco"

var
    i: inteiro
    numero, soma: real

inicio

    soma <- 0

    para i de 1 ate 5 faca

        escreva("Digite o valor ", i, ": ")
        leia(numero)

        soma <- soma + numero

    fimpara

    escreval("Soma = ", soma)

finalgoritmo
```

7.34 Calculando uma Média com Acumulador

Considere:

Ler cinco notas e calcular a média.

Precisamos:

1. somar as notas;
2. dividir a soma pela quantidade de notas.

```
algoritmo "media_cinco_notas"

var
    i: inteiro
    nota, soma, media: real

inicio
```

```
soma <- 0

para i de 1 ate 5 faca

    escreva("Digite a nota ", i, ": ")
    leia(nota)

    soma <- soma + nota

fimpara

media <- soma / 5

escreval("Media = ", media:4:2)

finalgoritmo
```

7.35 Contador e Acumulador no Mesmo Algoritmo

É possível utilizar os dois conceitos no mesmo problema. Considere: **Ler 10 notas, calcular a média da turma e contar quantos estudantes possuem nota maior ou igual a 7.**

```
algoritmo "estatisticas_turma"

var
    i, aprovados: inteiro
    nota, soma, media: real

inicio

    soma <- 0
    aprovados <- 0

    para i de 1 ate 10 faca

        escreva("Digite a nota ", i, ": ")
        leia(nota)

        soma <- soma + nota

        se nota >= 7 entao
            aprovados <- aprovados + 1
        fimse

    fimpara
```

```
media <- soma / 10

escreval("Media da turma = ", media:4:2)
escreval("Quantidade de aprovados = ", aprovados)

finalgoritmo
```

7.36 Valor Sentinela

Nem sempre sabemos quantos valores serão informados. Uma estratégia possível é utilizar um **valor sentinela**. Sentinela é um valor especial utilizado para indicar o encerramento da entrada. Considere: **Ler números até que seja informado o valor 0**. Podemos escrever:

```
algoritmo "sentinela"

var
    numero: real

inicio

    escreva("Digite um numero ou 0 para sair: ")
    leia(numero)

    enquanto numero <> 0 faca

        escreval("Valor informado: ", numero)

        escreva("Digite outro numero ou 0 para sair: ")
        leia(numero)

    fimenquanto

    escreval("Encerrado.")

finalgoritmo
```

Nesse exemplo, o valor 0 não é processado como um valor normal: ele indica que a repetição deve terminar.

7.37 Leitura Antes e Dentro do Laço

Observe novamente:

```
leia(numero)
```

```
enquanto numero <> 0 faca
```

```
    ...
```

```
    leia(numero)
```

```
fimenquanto
```

A primeira leitura permite que a condição seja avaliada antes da primeira iteração.

A leitura existente dentro do laço atualiza o valor para o próximo teste.

Essa organização é comum em repetições com sentinela controladas por **enquanto**.

7.38 Repetições Aninhadas

Assim como estruturas condicionais podem ser aninhadas, estruturas de repetição também podem ser colocadas dentro de outras repetições. Por exemplo:

```
para i de 1 ate 3 faca
```

```
    para j de 1 ate 4 faca
```

```
        escreval("i = ", i, "   j = ", j)
```

```
    fimpara
```

```
fimpara
```

O laço interno é executado completamente para cada iteração do laço externo.

7.39 Entendendo a Repetição Aninhada

No exemplo:

```
para i de 1 ate 3 faca
```

```
    para j de 1 ate 4 faca
```

```
        escreval(i, " ", j)
```

```
    fimpara
```

```
fimpara
```

```
    para:
```

$i = 1$

o valor de j assume:

1, 2, 3, 4

depois:

$i = 2$

e novamente:

$j = 1, 2, 3, 4$

e finalmente:

$i = 3$

com:

$j = 1, 2, 3, 4$

Assim, o comando interno é executado:

$3 \times 4 = 12$

vezes.

7.40 Exemplo – Tabuadas de 1 a 5

algoritmo "tabuadas"

var

 numero, multiplicador: inteiro

inicio

 para numero de 1 ate 5 faca

 escreval()

 escreval("Tabuada do ", numero)

 para multiplicador de 1 ate 10 faca

 escreval(

 numero,

 " x ",

 multiplicador,

 " = ",

 numero * multiplicador

)

 fimpara

```
fimpara  
  
fimalgoritmo
```

7.41 Erros Comuns

7.41.1 Esquecer de atualizar a variável de controle

Inadequado:

```
i <- 1  
  
enquanto i <= 10 faca  
    escreval(i)  
fimenquanto
```

O valor de *i* nunca muda. Adequado:

```
i <- 1  
  
enquanto i <= 10 faca  
    escreval(i)  
    i <- i + 1  
fimenquanto
```

7.41.2 Atualizar na direção errada

Considere:

```
i <- 1  
  
enquanto i <= 10 faca  
    escreval(i)  
    i <- i - 1  
fimenquanto
```

A variável ficará cada vez menor e continuará satisfazendo:

```
i <= 10  
  
produzindo um laço sem término esperado.
```

7.41.3 Inicializar o acumulador dentro do laço

Inadequado:

```
para i de 1 ate 5 faca  
    soma <- 0  
    leia(numero)  
    soma <- soma + numero  
fimpara
```

A cada iteração o valor acumulado seria reiniciado. Adequado:

```
soma <- 0  
  
para i de 1 ate 5 faca  
    leia(numero)  
    soma <- soma + numero  
fimpara
```

7.41.4 Inicializar o contador dentro do laço

Pelo mesmo motivo, o contador deve normalmente ser inicializado antes da repetição:

```
contador <- 0  
  
para i de 1 ate 10 faca  
    ...  
    se condicao entao  
        contador <- contador + 1  
    fimse  
fimpara
```

7.41.5 Confundir as condições de enquanto e repita

Observe:

```
enquanto resposta = "S" faca  
    significa:
```

repita enquanto a resposta for S.

Já:

repita

...

ate resposta = "N"

significa:

repita até a resposta ser N.

7.42 Comparação Geral das Estruturas

Tabela 7.5: Comparação das estruturas de repetição.

Estrutura	Teste	Execuções mínimas	Uso típico
enquanto	Antes do bloco	0	Quantidade não necessariamente conhecida e teste necessário antes da execução.
repita	Depois do bloco	1	O bloco precisa ser executado antes de testar a condição de saída.
para	Controle automático	0	Número ou intervalo de repetições conhecido.

7.43 Resumo do Capítulo

Neste capítulo estudamos estruturas que permitem executar repetidamente um conjunto de comandos. O comando:

enquanto...faca

realiza o teste antes da execução do bloco. Por isso, o bloco pode ser executado zero ou mais vezes. O comando:

repita...ate

realiza o teste depois da execução do bloco. Portanto, seus comandos são executados pelo menos uma vez. O comando:

para...faca

utiliza uma variável de controle e é especialmente adequado quando conhecemos a quantidade ou os limites das repetições. Também estudamos:

- inicialização;
- condição;
- incremento e decremento;

- contadores;
- acumuladores;
- valores sentinela;
- laços infinitos;
- repetições aninhadas.

7.44 Atividades

Questões conceituais

1. O que é uma estrutura de repetição?
2. O que significa o termo laço?
3. O que é uma iteração?
4. Qual é a principal característica do comando `enquanto...faca`?
5. Em que momento sua condição é avaliada?
6. Quantas vezes, no mínimo, o bloco de um `enquanto` pode ser executado?
7. Qual é a principal característica do `repita...ate`?
8. Quantas vezes, no mínimo, seu bloco é executado?
9. Qual é a diferença entre a condição utilizada no `enquanto` e a condição utilizada no `ate`?
10. Quando o comando `para...faca` é especialmente adequado?
11. O que é uma variável de controle?
12. Qual é a finalidade de um contador?
13. Qual é a finalidade de um acumulador?
14. O que é um laço infinito?
15. O que é um valor sentinela?

Preveja a Saída

Determine a saída dos algoritmos seguintes.

1.

```
i <- 1

enquanto i <= 4 faca
    escreva(i, " ")
    i <- i + 1
fimenquanto
```
2.

```
i <- 5

repita
    escreva(i, " ")
    i <- i - 1
ate i = 0
```
3.

```
para i de 2 ate 10 passo 2 faca
    escreva(i, " ")
fimpara
```
4.

```
para i de 5 ate 1 passo -1 faca
    escreva(i, " ")
fimpara
```

Utilizando enquanto

Construa algoritmos para:

1. apresentar os números de 1 até 20;
2. apresentar os números de 20 até 1;
3. apresentar os números pares de 2 até 100;
4. calcular a soma dos números de 1 até 10;
5. ler números enquanto eles forem positivos;
6. solicitar uma senha enquanto a senha informada estiver incorreta.

Utilizando repita

Construa algoritmos para:

1. apresentar os números de 1 até 10;
2. apresentar uma contagem regressiva de 10 até 1;
3. solicitar um número até que seja informado um valor positivo;
4. repetir uma soma até que o usuário escolha encerrar;
5. solicitar uma nota até que seja informado um valor entre 0 e 10.

Utilizando para

Construa algoritmos para:

1. apresentar os números de 1 até 100;
2. apresentar os números pares de 0 até 100;
3. apresentar os números de 100 até 1;
4. ler 10 números;
5. ler 10 números e calcular sua soma;
6. ler 10 números e calcular sua média;
7. apresentar a tabuada de um número informado pelo usuário.

Contadores

Construa algoritmos para:

1. ler 10 números e contar quantos são positivos;
2. ler 10 números e contar quantos são pares;
3. ler as notas de 20 estudantes e contar quantas são maiores ou iguais a 7;
4. ler 15 idades e contar quantas são maiores ou iguais a 18.

Acumuladores

Construa algoritmos para:

1. calcular a soma dos números de 1 até 100;
2. ler 10 valores e calcular sua soma;
3. ler cinco preços e calcular o valor total;
4. ler dez notas e calcular a média.

Contadores e Acumuladores

Construa um algoritmo que leia as notas de 10 estudantes e apresente:

- a soma das notas;
- a média da turma;
- a quantidade de notas maiores ou iguais a 7;
- a quantidade de notas inferiores a 7.

Encontre o Erro

Analise:

```
algoritmo "erro"

var
    i: inteiro

inicio

    i <- 1

    enquanto i <= 10 faca

        escreval(i)

    fimenquanto

finalgoritmo
```

Responda:

1. Qual problema ocorrerá?
2. Por que ele ocorre?
3. Qual comando precisa ser acrescentado?
4. Apresente a versão corrigida.

7.45 Desafio

Desafio – Análise de uma Turma

Uma turma possui 20 estudantes.
Para cada estudante deverá ser informada uma nota entre 0 e 10.
O algoritmo deverá:

1. ler as 20 notas;
2. calcular a soma das notas;
3. calcular a média da turma;
4. contar quantas notas são maiores ou iguais a 7;
5. contar quantas notas são inferiores a 7;
6. apresentar todos os resultados ao final.

Antes de escrever o algoritmo:

1. identifique quais variáveis funcionarão como contadores;
2. identifique qual variável funcionará como acumulador;
3. determine qual estrutura de repetição é mais apropriada;
4. construa o fluxograma;
5. escreva o algoritmo em Portugol;
6. realize um teste de mesa simplificado utilizando inicialmente apenas cinco notas.